NativePHP Mobile - Complete Implementation Guide

From Installation to Production

The Most Comprehensive Guide for Building Native Mobile Apps with Laravel

---

Table of Contents

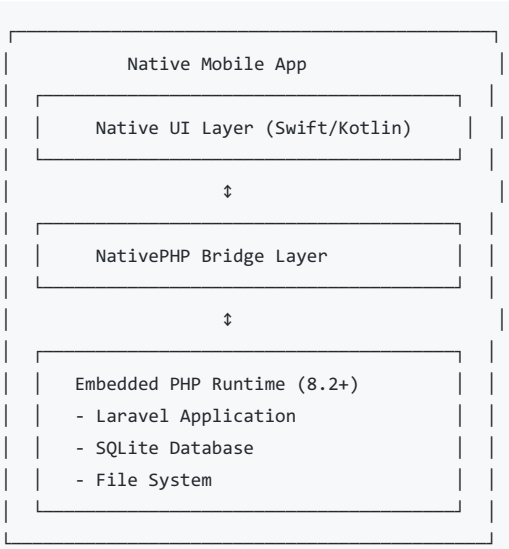
Part 1: Introduction & Architecture

What is NativePHP Mobile?

NativePHP Mobile is a framework that allows Laravel developers to build truly native mobile applications for iOS and Android. Unlike hybrid frameworks (like Ionic or Cordova), NativePHP embeds a complete PHP runtime directly into the native app, providing:

- **True Native Performance**: No web views, no JavaScript bridge delays
- **Direct Native API Access**: Camera, GPS, Bluetooth, etc.
- **Offline-First Architecture**: Apps work without internet
- **Laravel Ecosystem**: Use all your favorite Laravel packages

How It Works



Architecture Components

1. Native Layer

- Handles UI rendering (native components)
- Manages device permissions
- Executes native API calls

2. Bridge Layer

- Translates PHP calls to native code
- Handles events from native to PHP
- Manages data serialization

3. PHP Runtime Layer

- Your Laravel application
- Business logic and data processing
- Local SQLite database
- File operations

4. Optional API Layer

- Remote Laravel backend
- Real-time sync
- Multi-device data sharing

Part 2: Environment Setup & Installation

System Requirements Deep Dive

Why Java 17/21 and NOT Java 25?

Android's Gradle build system (version 8.13) is specifically tested with Java 17 LTS and Java 21 LTS. Java 25 introduces new APIs and deprecations that aren't compatible yet. You'll get errors like:

```
FAILURE: Build failed with an exception.
* What went wrong:
25.0.2
```

Installing Java 17 (Recommended)

Windows:

```
# Download from Adoptium
# https://adoptium.net/temurin/releases/?version=17

# After installation, set JAVA_HOME
setx JAVA_HOME "C:\Program Files\Eclipse Adoptium\jdk-17.0.9.9-hotspot"
setx PATH "%JAVA_HOME%\bin;%PATH%"

# Verify
java -version
# Should show: openjdk version "17.0.x"
```

macOS:

```
# Using Homebrew
brew install openjdk@17

# Link it
sudo ln -sfn /usr/local/opt/openjdk@17/libexec/openjdk.jdk /Library/Java/JavaVirtualMachines/openjdk-17.jdk

# Verify
java -version
```

Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install openjdk-17-jdk

# Verify
java -version
```

Complete Installation Process

Step 1: Create Laravel Project

```
# Create new Laravel project
composer create-project laravel/laravel mobile-app
cd mobile-app

# Install Laravel UI (optional but helpful for auth)
composer require laravel/ui
php artisan ui bootstrap --auth
npm install && npm run build
```

Step 2: Install NativePHP

```
# Install NativePHP Mobile package
composer require nativephp/mobile

# This installs:
# - nativephp/mobile (core package)
# - Dependencies for Android/iOS builds
# - CLI tools for building apps
```

Step 3: Initialize Platform

```
# For Android development
php artisan native:install android

# This creates:
# - nativephp/ directory (build files)
# - config/nativephp.php (configuration)
# - Android project structure
# - Gradle configuration files

# For iOS (macOS only)
php artisan native:install ios

# This creates:
# - iOS project in nativephp/ios/
# - Xcode workspace
# - CocoaPods configuration
```

Step 4: Understanding the Directory Structure

After installation, your project structure looks like:

```
mobile-app/
├── app/
│   ├── Http/
│   │   ├── Controllers/     # Your controllers
│   │   └── Livewire/        # Livewire components (recommended)
│   ├── Models/              # Eloquent models
│   └── Services/            # Business logic (API services, etc.)
├── config/
│   └── nativephp.php        # NativePHP configuration
├── database/
│   ├── migrations/          # Database schema
│   └── seeders/             # Test data
├── nativephp/
│   ├── android/             # Android project
│   │   ├── app/
│   │   └── build.gradle
│   └── ios/                 # iOS project (macOS only)
├── resources/
│   ├── views/               # Blade templates
│   └── js/                  # Frontend assets
├── routes/
│   ├── web.php              # Web routes
│   └── api.php              # API routes (for remote backend)
└── composer.json
```

Step 5: Configure the App

Edit `config/nativephp.php`:

```
<?php

return [
    /*
    |--------------------------------------------------------------------------
    | App ID
    |--------------------------------------------------------------------------
    | Unique identifier for your app (reverse domain notation)
    | Android: package name
    | iOS: bundle identifier
    */
    'app_id' => env('NATIVEPHP_APP_ID', 'com.yourcompany.mobileapp'),

    /*
    |--------------------------------------------------------------------------
    | App Version
    |--------------------------------------------------------------------------
    | Version number shown in app stores
    | Format: MAJOR.MINOR.PATCH (e.g., 1.0.0)
    */
    'app_version' => env('NATIVEPHP_APP_VERSION', '1.0.0'),

    /*
    |--------------------------------------------------------------------------
    | App Name
    |--------------------------------------------------------------------------
    | Display name shown to users
    */
    'name' => env('NATIVEPHP_APP_NAME', 'My Mobile App'),

    /*
    |--------------------------------------------------------------------------
    | Minimum SDK Versions
    |--------------------------------------------------------------------------
    */
    'android' => [
```

```
    'android' => [
        'minSdkVersion' => 24,      // Android 7.0+
        'targetSdkVersion' => 34,   // Android 14
        'compileSdkVersion' => 34,
    ],

    'ios' => [
        'minVersion' => '13.0',     // iOS 13+
    ],

    /*
    |--------------------------------------------------------------------------
    | Permissions
    |--------------------------------------------------------------------------
    | Enable only the permissions your app actually needs
    */
    'permissions' => [
        'camera' => false,
        'microphone' => false,
        'location' => false,
        'storage_read' => false,
        'storage_write' => false,
        // ... more permissions
    ],

    /*
    |--------------------------------------------------------------------------
    | App Icons
    |--------------------------------------------------------------------------
    */
    'icon' => [
        'path' => resource_path('images/icon.png'), // 1024x1024 PNG
    ],

    /*
    |--------------------------------------------------------------------------
    | Splash Screen
    |--------------------------------------------------------------------------
    */
    'splash' => [
        'path' => resource_path('images/splash.png'),
        'background_color' => '#FFFFFF',
    ],
];
```

Update `.env`:

```
 APP_NAME="My Mobile App"
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost

# NativePHP Configuration
NATIVEPHP_APP_ID=com.yourcompany.mobileapp
NATIVEPHP_APP_VERSION=1.0.0
NATIVEPHP_APP_NAME="My Mobile App"

# Database (SQLite for mobile)
DB_CONNECTION=sqlite
DB_DATABASE=database/database.sqlite

# API Configuration (if using remote backend)
API_URL=https://api.yourapp.com
API_TIMEOUT=30
```

Step 6: Create SQLite Database

```
# Create SQLite database file
touch database/database.sqlite

# Run migrations
php artisan migrate

# Seed with test data (optional)
php artisan db:seed
```

Step 7: Test the Setup

```
# Run the app in development mode
php artisan native:run android

# This will:
# 1. Build the Android app
# 2. Install it on connected device/emulator
# 3. Launch the app
# 4. Enable hot reload (changes reflect instantly)
```

Part 3: Your First Mobile App - Step by Step

Let's build a simple "Hello World" app with navigation to understand the flow.

Step 1: Install Livewire (Recommended for Mobile)

```
composer require livewire/livewire
```

Why Livewire?

- Real-time updates without page refresh
- Component-based architecture
- Perfect for mobile UI patterns

Step 2: Create Home Component

```
php artisan make:livewire Home
```

This creates:

- `app/Livewire/Home.php` (component class)
- `resources/views/livewire/home.blade.php` (view)

Edit `app/Livewire/Home.php`:

```php
<?php

namespace App\Livewire;

use Livewire\Component;
use Illuminate\Support\Facades\DB;

class Home extends Component
{
    public $greeting = "Welcome to NativePHP!";
    public $counter = 0;
    public $deviceInfo = [];

    public function mount()
    {
        // Get device information
        $this->deviceInfo = [
            'platform' => PHP_OS,
            'php_version' => PHP_VERSION,
            'laravel_version' => app()->version(),
            'database' => DB::connection()->getDatabaseName(),
        ];
    }

    public function increment()
    {
        $this->counter++;
    }

    public function render()
    {
        return view('livewire.home');
    }
}
```

Edit `resources/views/livewire/home.blade.php`:

```
<div class="container py-4">
    {{-- Header --}}
    <div class="text-center mb-4">
        <h1 class="display-4">{{ $greeting }}</h1>
        <p class="text-muted">Your First NativePHP Mobile App</p>
    </div>

    {{-- Counter Card --}}
    <div class="card mb-4">
        <div class="card-body text-center">
            <h2 class="card-title">Counter: {{ $counter }}</h2>
            <button wire:click="increment" class="btn btn-primary btn-lg">
                Increment
            </button>
        </div>
    </div>

    {{-- Device Info Card --}}
    <div class="card">
        <div class="card-header">
            <h5 class="mb-0">Device Information</h5>
        </div>
        <div class="card-body">
            <table class="table table-sm">
                <tbody>
                    @foreach($deviceInfo as $key => $value)
                    <tr>
                        <td><strong>{{ ucfirst(str_replace('_', ' ', $key)) }}</strong></td>
                        <td>{{ $value }}</td>
                    </tr>
                    @endforeach
                </tbody>
            </table>
        </div>
    </div>
</div>
```

Step 3: Create Main Layout

Create `resources/views/layouts/app.blade.php`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{{ config('app.name') }}</title>

    {{-- Bootstrap CSS --}}
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">

    {{-- Custom Mobile Styles --}}
    <style>
        body {
            font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen, Ubuntu, Cantarell, sans-serif;
            background-color: #f8f9fa;
        }

        .btn {
            border-radius: 8px;
            padding: 12px 24px;
            font-weight: 600;
        }

        .card {
            border-radius: 12px;
            border: none;
            box-shadow: 0 2px 8px rgba(0,0,0,0.1);
        }

        /* Safe area for notched devices */
        .container {
            padding-top: env(safe-area-inset-top);
            padding-bottom: env(safe-area-inset-bottom);
        }
    </style>

    @livewireStyles
</head>
<body>
    {{-- Navigation --}}
    <nav class="navbar navbar-light bg-white border-bottom">
        <div class="container">
            <span class="navbar-brand mb-0 h1">{{ config('app.name') }}</span>
        </div>
    </nav>

    {{-- Main Content --}}
    <main>
        {{ $slot }}
    </main>

    {{-- Bootstrap JS --}}
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>

    @livewireScripts
</body>
</html>
```

Step 4: Set Up Routes

Edit `routes/web.php`:

```
<?php

use Illuminate\Support\Facades\Route;
use App\Livewire\Home;

Route::get('/', Home::class)->name('home');
```

Step 5: Build and Test

```
# Build and run on Android
php artisan native:run android

# Build and run on iOS (macOS only)
php artisan native:run ios
```

You should see:

1. App launches on your device/emulator
2. Welcome screen with counter
3. Device information displayed
4. Counter increments when button is clicked

Part 4: Camera Plugin - Complete Implementation

Let's build a complete photo capture and management system.

Step 1: Install Camera Plugin

```
composer require nativephp/mobile-camera
```

Step 2: Enable Camera Permission

Edit `config/nativephp.php`:

```
'permissions' => [
    'camera' => true,  // Enable camera access
    'storage_write' => true,  // Enable saving photos
],
```

Step 3: Create Database Migration for Photos

```
php artisan make:migration create_photos_table
```

Edit the migration file:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('photos', function (Blueprint $table) {
            $table->id();
            $table->string('title')->nullable();
            $table->text('description')->nullable();
            $table->string('path'); // Local file path
            $table->string('filename');
            $table->integer('size'); // File size in bytes
            $table->string('mime_type'); // image/jpeg, image/png
            $table->integer('width')->nullable();
            $table->integer('height')->nullable();
            $table->timestamp('taken_at'); // When photo was taken
            $table->string('location')->nullable(); // GPS coordinates (if available)
            $table->boolean('is_synced')->default(false); // Sync status
            $table->string('remote_url')->nullable(); // URL after upload to server
            $table->timestamps();
            $table->softDeletes(); // For trash functionality
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('photos');
    }
};
```

Run migration:

```
php artisan migrate
```

Step 4: Create Photo Model

```
php artisan make:model Photo
```

Edit `app/Models/Photo.php`:

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;
use Illuminate\Support\Facades\Storage;

class Photo extends Model
{
    use HasFactory, SoftDeletes;

    protected $fillable = [
        'title',
        'description',
        'path',
        'filename',
```

```
        'size',
        'mime_type',
        'width',
        'height',
        'taken_at',
        'location',
        'is_synced',
        'remote_url',
    ];

    protected $casts = [
        'taken_at' => 'datetime',
        'is_synced' => 'boolean',
    ];

    /**
     * Get the full file path
     */
    public function getFullPathAttribute(): string
    {
        return storage_path('app/' . $this->path);
    }

    /**
     * Get file size in human readable format
     */
    public function getFileSizeAttribute(): string
    {
        $bytes = $this->size;
        $units = ['B', 'KB', 'MB', 'GB'];

        for ($i = 0; $bytes > 1024; $i++) {
            $bytes /= 1024;
        }

        return round($bytes, 2) . ' ' . $units[$i];
    }

    /**
     * Get photo dimensions as string
     */
    public function getDimensionsAttribute(): string
    {
        if ($this->width && $this->height) {
            return "{$this->width} × {$this->height}";
        }
        return 'Unknown';
    }

    /**
     * Scope for unsynced photos
     */
    public function scopeUnsynced($query)
    {
        return $query->where('is_synced', false);
    }

    /**
     * Delete photo file when model is deleted
     */
    protected static function booted()
    {
        static::deleted(function ($photo) {
            if (file_exists($photo->full_path)) {
                unlink($photo->full_path);
            }
```

```
        });
    }
}
```

Step 5: Create Photo Service

Create `app/Services/PhotoService.php`:

```php
<?php

namespace App\Services;

use App\Models\Photo;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Str;

class PhotoService
{
    /**
     * Save photo from camera
     */
    public function savePhoto(string $sourcePath, ?string $title = null): Photo
    {
        // Generate unique filename
        $filename = Str::uuid() . '.jpg';
        $storagePath = 'photos/' . date('Y/m/d');
        $fullPath = $storagePath . '/' . $filename;

        // Ensure directory exists
        Storage::makeDirectory($storagePath);

        // Copy file to app storage
        $destinationPath = storage_path('app/' . $fullPath);
        copy($sourcePath, $destinationPath);

        // Get image dimensions
        $imageInfo = getimagesize($destinationPath);

        // Create database record
        $photo = Photo::create([
            'title' => $title ?? 'Photo ' . now()->format('Y-m-d H:i'),
            'path' => $fullPath,
            'filename' => $filename,
            'size' => filesize($destinationPath),
            'mime_type' => $imageInfo['mime'] ?? 'image/jpeg',
            'width' => $imageInfo[0] ?? null,
            'height' => $imageInfo[1] ?? null,
            'taken_at' => now(),
        ]);

        return $photo;
    }

    /**
     * Update photo details
     */
    public function updatePhoto(Photo $photo, array $data): Photo
    {
        $photo->update($data);
        return $photo->fresh();
    }

    /**
     * Delete photo
```

```
     */
    public function deletePhoto(Photo $photo): bool
    {
        return $photo->delete();
    }

    /**
     * Get all photos
     */
    public function getAllPhotos()
    {
        return Photo::latest('taken_at')->get();
    }

    /**
     * Get photo by ID
     */
    public function getPhoto(int $id): ?Photo
    {
        return Photo::find($id);
    }

    /**
     * Compress image
     */
    public function compressImage(string $sourcePath, int $quality = 80): string
    {
        $image = imagecreatefromjpeg($sourcePath);
        $tempPath = sys_get_temp_dir() . '/' . Str::uuid() . '.jpg';
        imagejpeg($image, $tempPath, $quality);
        imagedestroy($image);

        return $tempPath;
    }
}
```

Step 6: Create Camera Livewire Component

```
php artisan make:livewire Camera/CameraCapture
```

Edit `app/Livewire/Camera/CameraCapture.php`:

```
<?php

namespace App\Livewire\Camera;

use Livewire\Component;
use Native\Mobile\Facades\Camera;
use Native\Mobile\Events\Camera\PhotoTaken;
use Native\Mobile\Events\Camera\VideoRecorded;
use Native\Mobile\Attributes\OnNative;
use App\Services\PhotoService;
use App\Models\Photo;

class CameraCapture extends Component
{
    public $photos = [];
    public $selectedPhoto = null;
    public $showPhotoModal = false;

    // Photo editing
    public $editingPhotoId = null;
    public $editTitle = '';
    public $editDescription = '';
```

```
protected $photoService;

public function boot(PhotoService $photoService)
{
    $this->photoService = $photoService;
}

public function mount()
{
    $this->loadPhotos();
}

/**
 * Load all photos from database
 */
public function loadPhotos()
{
    $this->photos = $this->photoService->getAllPhotos();
}

/**
 * Take a photo using device camera
 */
public function takePhoto()
{
    try {
        Camera::getPhoto();

        // Show loading indicator
        $this->dispatch('photo-capturing');

    } catch (\Exception $e) {
        session()->flash('error', 'Failed to open camera: ' . $e->getMessage());
    }
}

/**
 * Pick photo from gallery
 */
public function pickFromGallery()
{
    try {
        Camera::pickImages('gallery-pick', false); // false = single selection
        $this->dispatch('photo-picking');
    } catch (\Exception $e) {
        session()->flash('error', 'Failed to open gallery: ' . $e->getMessage());
    }
}

/**
 * Handle photo taken event from camera
 */
#[OnNative(PhotoTaken::class)]
public function handlePhotoTaken(string $path)
{
    try {
        // Save photo to database and storage
        $photo = $this->photoService->savePhoto($path);

        // Reload photos
        $this->loadPhotos();

        // Show success message
        session()->flash('success', 'Photo captured successfully!');
```

```
        // Optional: Show the photo immediately
        $this->viewPhoto($photo->id);

    } catch (\Exception $e) {
        session()->flash('error', 'Failed to save photo: ' . $e->getMessage());
    }
}

/**
 * View photo in modal
 */
public function viewPhoto($photoId)
{
    $this->selectedPhoto = $this->photoService->getPhoto($photoId);
    $this->showPhotoModal = true;
}

/**
 * Close photo modal
 */
public function closeModal()
{
    $this->showPhotoModal = false;
    $this->selectedPhoto = null;
}

/**
 * Start editing photo
 */
public function editPhoto($photoId)
{
    $photo = $this->photoService->getPhoto($photoId);

    if ($photo) {
        $this->editingPhotoId = $photo->id;
        $this->editTitle = $photo->title;
        $this->editDescription = $photo->description ?? '';
    }
}

/**
 * Save photo edits
 */
public function savePhotoEdits()
{
    $this->validate([
        'editTitle' => 'required|string|max:255',
        'editDescription' => 'nullable|string|max:1000',
    ]);

    $photo = $this->photoService->getPhoto($this->editingPhotoId);

    if ($photo) {
        $this->photoService->updatePhoto($photo, [
            'title' => $this->editTitle,
            'description' => $this->editDescription,
        ]);

        $this->loadPhotos();
        $this->cancelEdit();

        session()->flash('success', 'Photo updated successfully!');
    }
}
```

```
    /**
     * Cancel editing
     */
    public function cancelEdit()
    {
        $this->editingPhotoId = null;
        $this->editTitle = '';
        $this->editDescription = '';
    }

    /**
     * Delete photo
     */
    public function deletePhoto($photoId)
    {
        $photo = $this->photoService->getPhoto($photoId);

        if ($photo) {
            $this->photoService->deletePhoto($photo);
            $this->loadPhotos();
            $this->closeModal();

            session()->flash('success', 'Photo deleted successfully!');
        }
    }

    public function render()
    {
        return view('livewire.camera.camera-capture');
    }
}
```

Step 7: Create Camera View

Edit `resources/views/livewire/camera/camera-capture.blade.php`:

```
<div class="container py-4">
    {{-- Flash Messages --}}
    @if (session()->has('success'))
        <div class="alert alert-success alert-dismissible fade show" role="alert">
            {{ session('success') }}
            <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
        </div>
    @endif

    @if (session()->has('error'))
        <div class="alert alert-danger alert-dismissible fade show" role="alert">
            {{ session('error') }}
            <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
        </div>
    @endif

    {{-- Header --}}
    <div class="d-flex justify-content-between align-items-center mb-4">
        <h2>My Photos</h2>
        <span class="badge bg-primary">{{ count($photos) }} photos</span>
    </div>

    {{-- Action Buttons --}}
    <div class="row g-3 mb-4">
        <div class="col-6">
            <button wire:click="takePhoto" class="btn btn-primary w-100">
                <i class="bi bi-camera"></i> Take Photo
            </button>
```

```
        </div>
        <div class="col-6">
            <button wire:click="pickFromGallery" class="btn btn-outline-primary w-100">
                <i class="bi bi-images"></i> From Gallery
            </button>
        </div>
    </div>

    {{-- Photos Grid --}}
    @if (count($photos) > 0)
        <div class="row g-3">
            @foreach ($photos as $photo)
                <div class="col-4">
                    <div class="card h-100" wire:click="viewPhoto({{ $photo->id }})" style="cursor: pointer;">
                        <img src="{{ base64_encode(file_get_contents($photo->full_path)) }}"
                             class="card-img-top"
                             alt="{{ $photo->title }}"
                             style="height: 150px; object-fit: cover;">
                        <div class="card-body p-2">
                            <p class="card-text small text-truncate mb-0">{{ $photo->title }}</p>
                            <small class="text-muted">{{ $photo->taken_at->diffForHumans() }}</small>
                        </div>
                    </div>
                </div>
            @endforeach
        </div>
    @else
        <div class="text-center py-5">
            <i class="bi bi-camera" style="font-size: 4rem; color: #ccc;"></i>
            <p class="text-muted mt-3">No photos yet. Take your first photo!</p>
        </div>
    @endif

    {{-- Photo Detail Modal --}}
    @if ($showPhotoModal && $selectedPhoto)
        <div class="modal show d-block" tabindex="-1" style="background: rgba(0,0,0,0.5);">
            <div class="modal-dialog modal-fullscreen">
                <div class="modal-content">
                    <div class="modal-header">
                        <h5 class="modal-title">{{ $selectedPhoto->title }}</h5>
                        <button type="button" class="btn-close" wire:click="closeModal"></button>
                    </div>
                    <div class="modal-body p-0">
                        {{-- Full Image --}}
                        <img src="{{ base64_encode(file_get_contents($selectedPhoto->full_path)) }}"
                             class="w-100"
                             alt="{{ $selectedPhoto->title }}">

                        {{-- Photo Info --}}
                        <div class="p-3">
                            @if ($editingPhotoId === $selectedPhoto->id)
                                {{-- Edit Form --}}
                                <div class="mb-3">
                                    <label class="form-label">Title</label>
                                    <input type="text" wire:model="editTitle" class="form-control">
                                </div>
                                <div class="mb-3">
                                    <label class="form-label">Description</label>
                                    <textarea wire:model="editDescription" class="form-control" rows="3"></textarea>
                                </div>
                                <div class="d-flex gap-2">
                                    <button wire:click="savePhotoEdits" class="btn btn-primary">Save</button>
                                    <button wire:click="cancelEdit" class="btn btn-secondary">Cancel</button>
                                </div>
                            @else
                                {{-- View Mode --}}
```

```
                            <h6>{{ $selectedPhoto->title }}</h6>
                            @if ($selectedPhoto->description)
                                <p class="text-muted">{{ $selectedPhoto->description }}</p>
                            @endif

                            <div class="mt-3">
                                <small class="d-block"><strong>Size:</strong> {{ $selectedPhoto->file_size }}</small>
                                <small class="d-block"><strong>Dimensions:</strong> {{ $selectedPhoto->dimensions }}</small>
                                <small class="d-block"><strong>Taken:</strong> {{ $selectedPhoto->taken_at->format('M d, Y h:i A
                            </div>
                        @endif
                    </div>
                </div>
                <div class="modal-footer">
                    <button wire:click="editPhoto({{ $selectedPhoto->id }})" class="btn btn-outline-primary">
                        Edit
                    </button>
                    <button wire:click="deletePhoto({{ $selectedPhoto->id }})"
                            class="btn btn-outline-danger"
                            onclick="return confirm('Are you sure you want to delete this photo?')">
                        Delete
                    </button>
                </div>
            </div>
        </div>
    </div>
@endif
</div>

{{-- Bootstrap Icons --}}
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-icons.css">
```

Step 8: Add Camera Route

Edit `routes/web.php`:

```
use App\Livewire\Camera\CameraCapture;

Route::get('/camera', CameraCapture::class)->name('camera');
```

Step 9: Test Camera Feature

```
# Rebuild the app to include camera permissions
php artisan native:build android

# Run on device
php artisan native:run android
```

Now you have a fully functional camera system with:

- ✅ Take photos with device camera
- ✅ Pick photos from gallery
- ✅ Save photos to local database
- ✅ View photo grid
- ✅ View full-size photos
- ✅ Edit photo title and description
- ✅ Delete photos
- ✅ Display photo metadata

Part 5: File Management - Complete System

Let's build a complete file manager for documents and images.

Step 1: Install File Plugin

```
composer require nativephp/mobile-file
```

Step 2: Enable Storage Permissions

```
// config/nativephp.php
'permissions' => [
    'storage_read' => true,
    'storage_write' => true,
],
```

Step 3: Create Files Migration

```
php artisan make:migration create_files_table
```

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('files', function (Blueprint $table) {
            $table->id();
            $table->string('name'); // Original filename
            $table->string('display_name'); // User-friendly name
            $table->string('path'); // Storage path
            $table->string('extension'); // File extension
            $table->string('mime_type'); // MIME type
            $table->bigInteger('size'); // File size in bytes
            $table->string('category')->default('other'); // document, image, video, audio, other
            $table->text('description')->nullable();
            $table->json('metadata')->nullable(); // Additional file info
            $table->boolean('is_favorite')->default(false);
            $table->timestamps();
            $table->softDeletes();

            // Indexes for performance
            $table->index('category');
            $table->index('created_at');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('files');
    }
};
```

Run migration:

```
php artisan migrate
```

Step 4: Create File Model

```
php artisan make:model File
```

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;
use Illuminate\Support\Facades\Storage;

class File extends Model
{
    use SoftDeletes;

    protected $fillable = [
        'name',
        'display_name',
        'path',
        'extension',
        'mime_type',
        'size',
        'category',
        'description',
        'metadata',
        'is_favorite',
    ];

    protected $casts = [
        'metadata' => 'array',
        'is_favorite' => 'boolean',
    ];

    /**
     * Get full storage path
     */
    public function getFullPathAttribute(): string
    {
        return storage_path('app/' . $this->path);
    }

    /**
     * Get file size in human readable format
     */
    public function getHumanSizeAttribute(): string
    {
        $bytes = $this->size;
        $units = ['B', 'KB', 'MB', 'GB', 'TB'];

        for ($i = 0; $bytes > 1024; $i++) {
            $bytes /= 1024;
        }

        return round($bytes, 2) . ' ' . $units[$i];
    }

    /**
     * Get file icon based on type
     */
    public function getIconAttribute(): string
    {
        return match($this->category) {
            'document' => 'bi-file-earmark-text',
            'image' => 'bi-file-earmark-image',
            'video' => 'bi-file-earmark-play',
```

```
            'audio' => 'bi-file-earmark-music',
            'pdf' => 'bi-file-earmark-pdf',
            default => 'bi-file-earmark',
        };
    }

    /**
     * Determine category from mime type
     */
    public static function determineCategoryFromMime(string $mimeType): string
    {
        if (str_starts_with($mimeType, 'image/')) {
            return 'image';
        } elseif (str_starts_with($mimeType, 'video/')) {
            return 'video';
        } elseif (str_starts_with($mimeType, 'audio/')) {
            return 'audio';
        } elseif ($mimeType === 'application/pdf') {
            return 'pdf';
        } elseif (in_array($mimeType, [
            'application/msword',
            'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
            'application/vnd.ms-excel',
            'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet',
            'text/plain',
        ])) {
            return 'document';
        }

        return 'other';
    }

    /**
     * Scope for favorites
     */
    public function scopeFavorites($query)
    {
        return $query->where('is_favorite', true);
    }

    /**
     * Scope by category
     */
    public function scopeByCategory($query, string $category)
    {
        return $query->where('category', $category);
    }

    /**
     * Delete file when model is deleted
     */
    protected static function booted()
    {
        static::deleted(function ($file) {
            if (file_exists($file->full_path)) {
                unlink($file->full_path);
            }
        });
    }
}
```

Step 5: Create File Service

Create `app/Services/FileService.php`:

```php
<?php

namespace App\Services;

use App\Models\File;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Str;
use Illuminate\Http\UploadedFile;

class FileService
{
    /**
     * Save uploaded file
     */
    public function saveFile($sourceFile, ?string $displayName = null): File
    {
        // Determine if it's a path or UploadedFile
        if (is_string($sourceFile)) {
            $sourcePath = $sourceFile;
            $originalName = basename($sourcePath);
            $mimeType = mime_content_type($sourcePath);
            $size = filesize($sourcePath);
        } else {
            $sourcePath = $sourceFile->getRealPath();
            $originalName = $sourceFile->getClientOriginalName();
            $mimeType = $sourceFile->getMimeType();
            $size = $sourceFile->getSize();
        }

        // Generate storage path
        $extension = pathinfo($originalName, PATHINFO_EXTENSION);
        $filename = Str::uuid() . '.' . $extension;
        $storagePath = 'files/' . date('Y/m/d');
        $fullPath = $storagePath . '/' . $filename;

        // Ensure directory exists
        Storage::makeDirectory($storagePath);

        // Copy file
        $destinationPath = storage_path('app/' . $fullPath);
        copy($sourcePath, $destinationPath);

        // Determine category
        $category = File::determineCategoryFromMime($mimeType);

        // Extract metadata based on file type
        $metadata = $this->extractMetadata($destinationPath, $mimeType);

        // Create database record
        $file = File::create([
            'name' => $originalName,
            'display_name' => $displayName ?? pathinfo($originalName, PATHINFO_FILENAME),
            'path' => $fullPath,
            'extension' => $extension,
            'mime_type' => $mimeType,
            'size' => $size,
            'category' => $category,
            'metadata' => $metadata,
        ]);

        return $file;
    }

    /**
     * Extract file metadata
```

```
     */
    private function extractMetadata(string $filePath, string $mimeType): array
    {
        $metadata = [];

        // For images, get dimensions
        if (str_starts_with($mimeType, 'image/')) {
            $imageInfo = @getimagesize($filePath);
            if ($imageInfo) {
                $metadata['width'] = $imageInfo[0];
                $metadata['height'] = $imageInfo[1];
            }
        }

        // For videos, you could use FFmpeg here
        // For PDFs, you could extract page count

        return $metadata;
    }

    /**
     * Update file
     */
    public function updateFile(File $file, array $data): File
    {
        $file->update($data);
        return $file->fresh();
    }

    /**
     * Delete file
     */
    public function deleteFile(File $file): bool
    {
        return $file->delete();
    }

    /**
     * Toggle favorite
     */
    public function toggleFavorite(File $file): File
    {
        $file->update(['is_favorite' => !$file->is_favorite]);
        return $file->fresh();
    }

    /**
     * Get files by category
     */
    public function getFilesByCategory(string $category)
    {
        return File::byCategory($category)->latest()->get();
    }

    /**
     * Get all files
     */
    public function getAllFiles()
    {
        return File::latest()->get();
    }

    /**
     * Get favorites
     */
    public function getFavorites()
```

```
    public function getFavorites()
    {
        return File::favorites()->latest()->get();
    }

    /**
     * Search files
     */
    public function searchFiles(string $query)
    {
        return File::where('display_name', 'like', "%{$query}%")
            ->orWhere('name', 'like', "%{$query}%")
            ->orWhere('description', 'like', "%{$query}%")
            ->latest()
            ->get();
    }

    /**
     * Get total storage used
     */
    public function getTotalStorageUsed(): int
    {
        return File::sum('size');
    }

    /**
     * Get storage used by category
     */
    public function getStorageByCategory(): array
    {
        return File::selectRaw('category, SUM(size) as total_size')
            ->groupBy('category')
            ->get()
            ->mapWithKeys(function ($item) {
                return [$item->category => $item->total_size];
            })
            ->toArray();
    }
}
```

Step 6: Create File Manager Component

```
php artisan make:livewire FileManager
```

```
<?php

namespace App\Livewire;

use Livewire\Component;
use Livewire\WithFileUploads;
use Native\Mobile\Facades\File as NativeFile;
use App\Services\FileService;
use App\Models\File;

class FileManager extends Component
{
    use WithFileUploads;

    public $files = [];
    public $currentCategory = 'all';
    public $searchQuery = '';
    public $selectedFile = null;
    public $showFileModal = false;
    public $showUploadModal = false;
```

```
    // Edit file
    public $editingFileId = null;
    public $editDisplayName = '';
    public $editDescription = '';

    // Stats
    public $totalSize = 0;
    public $fileCount = 0;

    protected $fileService;

    public function boot(FileService $fileService)
    {
        $this->fileService = $fileService;
    }

    public function mount()
    {
        $this->loadFiles();
        $this->loadStats();
    }

    /**
     * Load files
     */
    public function loadFiles()
    {
        if ($this->currentCategory === 'all') {
            $this->files = $this->fileService->getAllFiles();
        } elseif ($this->currentCategory === 'favorites') {
            $this->files = $this->fileService->getFavorites();
        } else {
            $this->files = $this->fileService->getFilesByCategory($this->currentCategory);
        }
    }

    /**
     * Load statistics
     */
    public function loadStats()
    {
        $this->totalSize = $this->fileService->getTotalStorageUsed();
        $this->fileCount = File::count();
    }

    /**
     * Change category filter
     */
    public function setCategory($category)
    {
        $this->currentCategory = $category;
        $this->loadFiles();
    }

    /**
     * Search files
     */
    public function search()
    {
        if (empty($this->searchQuery)) {
            $this->loadFiles();
        } else {
            $this->files = $this->fileService->searchFiles($this->searchQuery);
        }
```

```
    }

    /**
     * Clear search
     */
    public function clearSearch()
    {
        $this->searchQuery = '';
        $this->loadFiles();
    }

    /**
     * Pick file from device
     */
    public function pickFile()
    {
        try {
            // This will open the file picker
            // You'll need to handle the selected file
            $this->showUploadModal = true;
        } catch (\Exception $e) {
            session()->flash('error', 'Failed to open file picker: ' . $e->getMessage());
        }
    }

    /**
     * View file details
     */
    public function viewFile($fileId)
    {
        $this->selectedFile = File::find($fileId);
        $this->showFileModal = true;
    }

    /**
     * Close modal
     */
    public function closeModal()
    {
        $this->showFileModal = false;
        $this->selectedFile = null;
    }

    /**
     * Toggle favorite
     */
    public function toggleFavorite($fileId)
    {
        $file = File::find($fileId);
        if ($file) {
            $this->fileService->toggleFavorite($file);
            $this->loadFiles();
        }
    }

    /**
     * Start editing file
     */
    public function editFile($fileId)
    {
        $file = File::find($fileId);
        if ($file) {
            $this->editingFileId = $file->id;
            $this->editDisplayName = $file->display_name;
            $this->editDescription = $file->description ?? '';
        }
```

```
    }

    /**
     * Save file edits
     */
    public function saveFileEdits()
    {
        $this->validate([
            'editDisplayName' => 'required|string|max:255',
            'editDescription' => 'nullable|string|max:1000',
        ]);

        $file = File::find($this->editingFileId);
        if ($file) {
            $this->fileService->updateFile($file, [
                'display_name' => $this->editDisplayName,
                'description' => $this->editDescription,
            ]);

            $this->loadFiles();
            $this->cancelEdit();
            session()->flash('success', 'File updated successfully!');
        }
    }

    /**
     * Cancel editing
     */
    public function cancelEdit()
    {
        $this->editingFileId = null;
        $this->editDisplayName = '';
        $this->editDescription = '';
    }

    /**
     * Delete file
     */
    public function deleteFile($fileId)
    {
        $file = File::find($fileId);
        if ($file) {
            $this->fileService->deleteFile($file);
            $this->loadFiles();
            $this->loadStats();
            $this->closeModal();
            session()->flash('success', 'File deleted successfully!');
        }
    }

    /**
     * Download/Open file
     */
    public function openFile($fileId)
    {
        $file = File::find($fileId);
        if ($file && file_exists($file->full_path)) {
            // For mobile, we can trigger native file opening
            // or display in WebView depending on file type
            $this->dispatch('open-file', path: $file->full_path);
        }
    }

    public function render()
    {
```

```
        return view('livewire.file-manager');
    }
}
```

Step 7: Create File Manager View

Create `resources/views/livewire/file-manager.blade.php`:

```
<div class="container py-4">
    {{-- Flash Messages --}}
    @if (session()->has('success'))
        <div class="alert alert-success alert-dismissible fade show">
            {{ session('success') }}
            <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
        </div>
    @endif

    {{-- Header with Stats --}}
    <div class="card mb-4">
        <div class="card-body">
            <div class="row text-center">
                <div class="col-6">
                    <h3 class="mb-0">{{ $fileCount }}</h3>
                    <small class="text-muted">Total Files</small>
                </div>
                <div class="col-6">
                    <h3 class="mb-0">{{ number_format($totalSize / 1024 / 1024, 2) }} MB</h3>
                    <small class="text-muted">Storage Used</small>
                </div>
            </div>
        </div>
    </div>

    {{-- Search Bar --}}
    <div class="input-group mb-3">
        <input type="text"
               wire:model.live="searchQuery"
               class="form-control"
               placeholder="Search files...">
        @if ($searchQuery)
            <button wire:click="clearSearch" class="btn btn-outline-secondary">
                <i class="bi bi-x"></i>
            </button>
        @endif
    </div>

    {{-- Category Tabs --}}
    <ul class="nav nav-pills mb-4" style="overflow-x: auto; white-space: nowrap;">
        <li class="nav-item">
            <a class="nav-link {{ $currentCategory === 'all' ? 'active' : '' }}"
               wire:click="setCategory('all')"
               href="#">All</a>
        </li>
        <li class="nav-item">
            <a class="nav-link {{ $currentCategory === 'favorites' ? 'active' : '' }}"
               wire:click="setCategory('favorites')"
               href="#">Favorites</a>
        </li>
        <li class="nav-item">
            <a class="nav-link {{ $currentCategory === 'document' ? 'active' : '' }}"
               wire:click="setCategory('document')"
               href="#">Documents</a>
        </li>
        <li class="nav-item">
            <a class="nav-link {{ $currentCategory === 'image' ? 'active' : '' }}"
```

```
            wire:click="setCategory('image')"
            href="#">Images</a>
    </li>
    <li class="nav-item">
        <a class="nav-link {{ $currentCategory === 'video' ? 'active' : '' }}"
            wire:click="setCategory('video')"
            href="#">Videos</a>
    </li>
    <li class="nav-item">
        <a class="nav-link {{ $currentCategory === 'audio' ? 'active' : '' }}"
            wire:click="setCategory('audio')"
            href="#">Audio</a>
    </li>
</ul>

{{-- Files List --}}
@if (count($files) > 0)
    <div class="list-group mb-4">
        @foreach ($files as $file)
            <div class="list-group-item list-group-item-action"
                 wire:click="viewFile({{ $file->id }})"
                 style="cursor: pointer;">
                <div class="d-flex align-items-center">
                    {{-- File Icon --}}
                    <div class="me-3">
                        <i class="{{ $file->icon }}" style="font-size: 2rem; color: #6c757d;"></i>
                    </div>

                    {{-- File Info --}}
                    <div class="flex-grow-1">
                        <div class="d-flex justify-content-between align-items-start">
                            <div>
                                <h6 class="mb-1">{{ $file->display_name }}</h6>
                                <small class="text-muted">
                                    {{ $file->human_size }} • {{ $file->extension }} •
                                    {{ $file->created_at->diffForHumans() }}
                                </small>
                            </div>

                            {{-- Favorite Icon --}}
                            <button wire:click.stop="toggleFavorite({{ $file->id }})"
                                    class="btn btn-sm btn-link">
                                <i class="bi bi-star{{ $file->is_favorite ? '-fill text-warning' : '' }}"></i>
                            </button>
                        </div>
                    </div>
                </div>
            </div>
        @endforeach
    </div>
@else
    <div class="text-center py-5">
        <i class="bi bi-folder2-open" style="font-size: 4rem; color: #ccc;"></i>
        <p class="text-muted mt-3">
            @if ($searchQuery)
                No files found matching "{{ $searchQuery }}"
            @else
                No files in this category
            @endif
        </p>
    </div>
@endif

{{-- Upload Button (Fixed) --}}
<div class="position-fixed bottom-0 end-0 p-4">
```

```
    <button wire:click="pickFile" class="btn btn-primary btn-lg rounded-circle" style="width: 60px; height: 60px;">
        <i class="bi bi-plus-lg"></i>
    </button>
</div>

{{-- File Detail Modal --}}
@if ($showFileModal && $selectedFile)
    <div class="modal show d-block" tabindex="-1" style="background: rgba(0,0,0,0.5);">
        <div class="modal-dialog">
            <div class="modal-content">
                <div class="modal-header">
                    <h5 class="modal-title">{{ $selectedFile->display_name }}</h5>
                    <button type="button" class="btn-close" wire:click="closeModal"></button>
                </div>
                <div class="modal-body">
                    {{-- File Preview --}}
                    @if ($selectedFile->category === 'image')
                        <img src="data:{{ $selectedFile->mime_type }};base64,{{ base64_encode(file_get_contents($selectedFile->f
                             class="img-fluid mb-3 rounded">
                    @else
                        <div class="text-center py-4">
                            <i class="{{ $selectedFile->icon }}" style="font-size: 5rem; color: #6c757d;"></i>
                        </div>
                    @endif

                    @if ($editingFileId === $selectedFile->id)
                        {{-- Edit Form --}}
                        <div class="mb-3">
                            <label class="form-label">Display Name</label>
                            <input type="text" wire:model="editDisplayName" class="form-control">
                        </div>
                        <div class="mb-3">
                            <label class="form-label">Description</label>
                            <textarea wire:model="editDescription" class="form-control" rows="3"></textarea>
                        </div>
                    @else
                        {{-- File Details --}}
                        @if ($selectedFile->description)
                            <p class="text-muted">{{ $selectedFile->description }}</p>
                        @endif

                        <table class="table table-sm">
                            <tbody>
                                <tr>
                                    <td><strong>Name</strong></td>
                                    <td>{{ $selectedFile->name }}</td>
                                </tr>
                                <tr>
                                    <td><strong>Size</strong></td>
                                    <td>{{ $selectedFile->human_size }}</td>
                                </tr>
                                <tr>
                                    <td><strong>Type</strong></td>
                                    <td>{{ $selectedFile->mime_type }}</td>
                                </tr>
                                <tr>
                                    <td><strong>Category</strong></td>
                                    <td><span class="badge bg-secondary">{{ ucfirst($selectedFile->category) }}</span></td>
                                </tr>
                                <tr>
                                    <td><strong>Created</strong></td>
                                    <td>{{ $selectedFile->created_at->format('M d, Y h:i A') }}</td>
                                </tr>
                                @if ($selectedFile->metadata && isset($selectedFile->metadata['width']))
                                    <tr>
                                        <td><strong>Dimensions</strong></td>
```

```
                                    <td><strong>Dimensions</strong></td>
                                    <td>{{ $selectedFile->metadata['width'] }} × {{ $selectedFile->metadata['height'] }}</td
                                </tr>
                            @endif
                        </tbody>
                    </table>
                @endif
            </div>
            <div class="modal-footer">
                @if ($editingFileId === $selectedFile->id)
                    <button wire:click="saveFileEdits" class="btn btn-primary">Save</button>
                    <button wire:click="cancelEdit" class="btn btn-secondary">Cancel</button>
                @else
                    <button wire:click="openFile({{ $selectedFile->id }})" class="btn btn-primary">
                        Open
                    </button>
                    <button wire:click="editFile({{ $selectedFile->id }})" class="btn btn-outline-primary">
                        Edit
                    </button>
                    <button wire:click="deleteFile({{ $selectedFile->id }})"
                            class="btn btn-outline-danger"
                            onclick="return confirm('Are you sure?')">
                        Delete
                    </button>
                @endif
            </div>
        </div>
    </div>
</div>
@endif
</div>

<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-icons.css">
```

Step 8: Add Route

```
// routes/web.php
use App\Livewire\FileManager;

Route::get('/files', FileManager::class)->name('files');
```

This file management system provides:

- ✅ Upload files from device
- ✅ Categorize files automatically
- ✅ Search functionality
- ✅ Favorite files
- ✅ View file details and metadata
- ✅ Edit file names and descriptions
- ✅ Delete files
- ✅ Storage statistics
- ✅ Category filtering

Would you like me to continue with the remaining parts (Audio Recording, Scanner, Network, API Integration, Real-World Examples, and Publishing)? This is getting quite extensive, so I can create multiple files or continue with the single comprehensive guide.

Part 6: Audio Recording - Complete Implementation

Overview

Build a voice notes app with recording, playback, and organization features.

Step 1: Install Microphone Plugin

```
composer require nativephp/mobile-microphone
```

Step 2: Enable Permissions

```
// config/nativephp.php
'permissions' => [
    'microphone' => true,
    'microphone_background' => false, // Set true if you want background recording
    'storage_write' => true,
],
```

Step 3: Create Audio Recordings Migration

```
php artisan make:migration create_audio_recordings_table
```

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('audio_recordings', function (Blueprint $table) {
            $table->id();
            $table->string('title');
            $table->text('notes')->nullable();
            $table->string('path'); // Storage path
            $table->string('filename');
            $table->integer('duration'); // Duration in seconds
            $table->bigInteger('size'); // File size in bytes
            $table->string('format'); // m4a, mp3, etc.
            $table->timestamp('recorded_at');
            $table->boolean('is_favorite')->default(false);
            $table->json('tags')->nullable(); // Array of tags
            $table->timestamps();
            $table->softDeletes();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('audio_recordings');
    }
};
```

```
php artisan migrate
```

Step 4: Create AudioRecording Model

```
php artisan make:model AudioRecording
```

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
```

```php
use Illuminate\Database\Eloquent\SoftDeletes;

class AudioRecording extends Model
{
    use SoftDeletes;

    protected $fillable = [
        'title',
        'notes',
        'path',
        'filename',
        'duration',
        'size',
        'format',
        'recorded_at',
        'is_favorite',
        'tags',
    ];

    protected $casts = [
        'recorded_at' => 'datetime',
        'is_favorite' => 'boolean',
        'tags' => 'array',
    ];

    /**
     * Get full file path
     */
    public function getFullPathAttribute(): string
    {
        return storage_path('app/' . $this->path);
    }

    /**
     * Get formatted duration
     */
    public function getFormattedDurationAttribute(): string
    {
        $minutes = floor($this->duration / 60);
        $seconds = $this->duration % 60;
        return sprintf('%02d:%02d', $minutes, $seconds);
    }

    /**
     * Get file size in human readable format
     */
    public function getHumanSizeAttribute(): string
    {
        $bytes = $this->size;
        $units = ['B', 'KB', 'MB', 'GB'];

        for ($i = 0; $bytes > 1024; $i++) {
            $bytes /= 1024;
        }

        return round($bytes, 2) . ' ' . $units[$i];
    }

    /**
     * Scope for favorites
     */
    public function scopeFavorites($query)
    {
        return $query->where('is_favorite', true);
    }
```

```
    /**
     * Delete audio file when model is deleted
     */
    protected static function booted()
    {
        static::deleted(function ($recording) {
            if (file_exists($recording->full_path)) {
                unlink($recording->full_path);
            }
        });
    }
}
```

Step 5: Create Audio Service

Create `app/Services/AudioService.php`:

```
<?php

namespace App\Services;

use App\Models\AudioRecording;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Str;

class AudioService
{
    /**
     * Save audio recording
     */
    public function saveRecording(string $sourcePath, ?string $title = null): AudioRecording
    {
        // Generate filename
        $filename = Str::uuid() . '.m4a';
        $storagePath = 'audio/' . date('Y/m/d');
        $fullPath = $storagePath . '/' . $filename;

        // Ensure directory exists
        Storage::makeDirectory($storagePath);

        // Copy file
        $destinationPath = storage_path('app/' . $fullPath);
        copy($sourcePath, $destinationPath);

        // Get file info
        $filesize = filesize($destinationPath);
        $duration = $this->getAudioDuration($destinationPath);

        // Create database record
        $recording = AudioRecording::create([
            'title' => $title ?? 'Recording ' . now()->format('M d, Y h:i A'),
            'path' => $fullPath,
            'filename' => $filename,
            'duration' => $duration,
            'size' => $filesize,
            'format' => 'm4a',
            'recorded_at' => now(),
        ]);

        return $recording;
    }

    /**
     * Get audio duration (approximation for M4A)
```

```
     */
    private function getAudioDuration(string $filePath): int
    {
        // For accurate duration, you'd use FFmpeg or similar
        // This is a simple approximation based on file size
        // M4A typically has ~128kbps bitrate
        $filesize = filesize($filePath);
        $bitrate = 128 * 1024; // bits per second
        return (int) ($filesize * 8 / $bitrate);
    }

    /**
     * Update recording
     */
    public function updateRecording(AudioRecording $recording, array $data): AudioRecording
    {
        $recording->update($data);
        return $recording->fresh();
    }

    /**
     * Delete recording
     */
    public function deleteRecording(AudioRecording $recording): bool
    {
        return $recording->delete();
    }

    /**
     * Toggle favorite
     */
    public function toggleFavorite(AudioRecording $recording): AudioRecording
    {
        $recording->update(['is_favorite' => !$recording->is_favorite]);
        return $recording->fresh();
    }

    /**
     * Add tag to recording
     */
    public function addTag(AudioRecording $recording, string $tag): AudioRecording
    {
        $tags = $recording->tags ?? [];
        if (!in_array($tag, $tags)) {
            $tags[] = $tag;
            $recording->update(['tags' => $tags]);
        }
        return $recording->fresh();
    }

    /**
     * Remove tag from recording
     */
    public function removeTag(AudioRecording $recording, string $tag): AudioRecording
    {
        $tags = $recording->tags ?? [];
        $tags = array_values(array_diff($tags, [$tag]));
        $recording->update(['tags' => $tags]);
        return $recording->fresh();
    }

    /**
     * Get all recordings
     */
    public function getAllRecordings()
```

```
    {
        return AudioRecording::latest('recorded_at')->get();
    }

    /**
     * Get favorites
     */
    public function getFavorites()
    {
        return AudioRecording::favorites()->latest('recorded_at')->get();
    }

    /**
     * Search recordings
     */
    public function searchRecordings(string $query)
    {
        return AudioRecording::where('title', 'like', "%{$query}%")
            ->orWhere('notes', 'like', "%{$query}%")
            ->latest('recorded_at')
            ->get();
    }

    /**
     * Get recordings by tag
     */
    public function getRecordingsByTag(string $tag)
    {
        return AudioRecording::whereJsonContains('tags', $tag)
            ->latest('recorded_at')
            ->get();
    }
}
```

Step 6: Create Voice Recorder Component

```
php artisan make:livewire VoiceRecorder
```

```
<?php

namespace App\Livewire;

use Livewire\Component;
use Native\Mobile\Facades\Microphone;
use Native\Mobile\Events\Microphone\MicrophoneRecorded;
use Native\Mobile\Attributes\OnNative;
use App\Services\AudioService;
use App\Models\AudioRecording;

class VoiceRecorder extends Component
{
    public $recordings = [];
    public $isRecording = false;
    public $recordingTime = 0;
    public $selectedRecording = null;
    public $showRecordingModal = false;

    // Edit recording
    public $editingRecordingId = null;
    public $editTitle = '';
    public $editNotes = '';
    public $editTags = '';

    // Filters
```

```
    // Filters
    public $showFavoritesOnly = false;
    public $searchQuery = '';

    protected $audioService;

    public function boot(AudioService $audioService)
    {
        $this->audioService = $audioService;
    }

    public function mount()
    {
        $this->loadRecordings();
    }

    /**
     * Load recordings
     */
    public function loadRecordings()
    {
        if ($this->showFavoritesOnly) {
            $this->recordings = $this->audioService->getFavorites();
        } elseif ($this->searchQuery) {
            $this->recordings = $this->audioService->searchRecordings($this->searchQuery);
        } else {
            $this->recordings = $this->audioService->getAllRecordings();
        }
    }

    /**
     * Start recording
     */
    public function startRecording()
    {
        try {
            Microphone::record()
                ->id('voice-note-' . time())
                ->start();

            $this->isRecording = true;
            $this->recordingTime = 0;

            // Start timer
            $this->dispatch('start-timer');

        } catch (\Exception $e) {
            session()->flash('error', 'Failed to start recording: ' . $e->getMessage());
        }
    }

    /**
     * Stop recording
     */
    public function stopRecording()
    {
        try {
            Microphone::stop();
            $this->isRecording = false;
            $this->dispatch('stop-timer');
        } catch (\Exception $e) {
            session()->flash('error', 'Failed to stop recording: ' . $e->getMessage());
        }
    }

    /**
```

```
     * Pause recording
     */
    public function pauseRecording()
    {
        try {
            Microphone::pause();
            $this->dispatch('pause-timer');
        } catch (\Exception $e) {
            session()->flash('error', 'Failed to pause recording: ' . $e->getMessage());
        }
    }

    /**
     * Resume recording
     */
    public function resumeRecording()
    {
        try {
            Microphone::resume();
            $this->dispatch('resume-timer');
        } catch (\Exception $e) {
            session()->flash('error', 'Failed to resume recording: ' . $e->getMessage());
        }
    }

    /**
     * Handle recording completed
     */
    #[OnNative(MicrophoneRecorded::class)]
    public function handleRecordingCompleted(string $path, string $mimeType, ?string $id)
    {
        try {
            // Save recording
            $recording = $this->audioService->saveRecording($path);

            // Reload recordings
            $this->loadRecordings();

            // Reset recording state
            $this->isRecording = false;
            $this->recordingTime = 0;

            // Show success message
            session()->flash('success', 'Recording saved successfully!');

            // Open the recording for editing
            $this->viewRecording($recording->id);

        } catch (\Exception $e) {
            session()->flash('error', 'Failed to save recording: ' . $e->getMessage());
        }
    }

    /**
     * View recording details
     */
    public function viewRecording($recordingId)
    {
        $this->selectedRecording = AudioRecording::find($recordingId);
        $this->showRecordingModal = true;
    }

    /**
     * Close modal
     */
```

```
public function closeModal()
{
    $this->showRecordingModal = false;
    $this->selectedRecording = null;
    $this->cancelEdit();
}

/**
 * Toggle favorite
 */
public function toggleFavorite($recordingId)
{
    $recording = AudioRecording::find($recordingId);
    if ($recording) {
        $this->audioService->toggleFavorite($recording);
        $this->loadRecordings();
    }
}

/**
 * Toggle favorites filter
 */
public function toggleFavoritesFilter()
{
    $this->showFavoritesOnly = !$this->showFavoritesOnly;
    $this->loadRecordings();
}

/**
 * Search recordings
 */
public function updatedSearchQuery()
{
    $this->loadRecordings();
}

/**
 * Start editing
 */
public function editRecording($recordingId)
{
    $recording = AudioRecording::find($recordingId);
    if ($recording) {
        $this->editingRecordingId = $recording->id;
        $this->editTitle = $recording->title;
        $this->editNotes = $recording->notes ?? '';
        $this->editTags = $recording->tags ? implode(', ', $recording->tags) : '';
    }
}

/**
 * Save edits
 */
public function saveRecordingEdits()
{
    $this->validate([
        'editTitle' => 'required|string|max:255',
        'editNotes' => 'nullable|string|max:5000',
        'editTags' => 'nullable|string|max:500',
    ]);

    $recording = AudioRecording::find($this->editingRecordingId);
    if ($recording) {
        // Parse tags
        $tags = array_filter(array_map('trim', explode(',', $this->editTags)));
```

```
            $this->audioService->updateRecording($recording, [
                'title' => $this->editTitle,
                'notes' => $this->editNotes,
                'tags' => $tags,
            ]);

            $this->loadRecordings();
            $this->cancelEdit();

            session()->flash('success', 'Recording updated successfully!');
        }
    }

    /**
     * Cancel editing
     */
    public function cancelEdit()
    {
        $this->editingRecordingId = null;
        $this->editTitle = '';
        $this->editNotes = '';
        $this->editTags = '';
    }

    /**
     * Delete recording
     */
    public function deleteRecording($recordingId)
    {
        $recording = AudioRecording::find($recordingId);
        if ($recording) {
            $this->audioService->deleteRecording($recording);
            $this->loadRecordings();
            $this->closeModal();

            session()->flash('success', 'Recording deleted successfully!');
        }
    }

    /**
     * Play recording (trigger native audio player)
     */
    public function playRecording($recordingId)
    {
        $recording = AudioRecording::find($recordingId);
        if ($recording && file_exists($recording->full_path)) {
            $this->dispatch('play-audio', path: $recording->full_path);
        }
    }

    public function render()
    {
        return view('livewire.voice-recorder');
    }
}
```

Step 7: Create Voice Recorder View

Create `resources/views/livewire/voice-recorder.blade.php`:

```
<div class="container py-4">
    {{-- Flash Messages --}}
    @if (session()->has('success'))
        <div class="alert alert-success alert-dismissible fade show">
```

```
        {{ session('success') }}
        <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    </div>
@endif

@if (session()->has('error'))
    <div class="alert alert-danger alert-dismissible fade show">
        {{ session('error') }}
        <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    </div>
@endif

{{-- Recording Controls --}}
<div class="card mb-4">
    <div class="card-body text-center">
        @if ($isRecording)
            {{-- Recording in Progress --}}
            <div class="mb-3">
                <div class="spinner-grow text-danger" role="status">
                    <span class="visually-hidden">Recording...</span>
                </div>
            </div>
            <h3 class="mb-3" id="recording-timer">00:00</h3>
            <div class="d-flex justify-content-center gap-3">
                <button wire:click="pauseRecording" class="btn btn-warning btn-lg">
                    <i class="bi bi-pause-fill"></i> Pause
                </button>
                <button wire:click="stopRecording" class="btn btn-danger btn-lg">
                    <i class="bi bi-stop-fill"></i> Stop
                </button>
            </div>
        @else
            {{-- Start Recording --}}
            <button wire:click="startRecording" class="btn btn-danger btn-lg rounded-circle"
                    style="width: 100px; height: 100px;">
                <i class="bi bi-mic-fill" style="font-size: 2rem;"></i>
            </button>
            <p class="mt-3 text-muted">Tap to start recording</p>
        @endif
    </div>
</div>

{{-- Search and Filter --}}
<div class="row mb-3">
    <div class="col-8">
        <input type="text"
               wire:model.live="searchQuery"
               class="form-control"
               placeholder="Search recordings...">
    </div>
    <div class="col-4">
        <button wire:click="toggleFavoritesFilter"
                class="btn btn-outline-warning w-100">
            <i class="bi bi-star{{ $showFavoritesOnly ? '-fill' : '' }}"></i>
        </button>
    </div>
</div>

{{-- Recordings List --}}
<h5 class="mb-3">My Recordings ({{ count($recordings) }})</h5>

@if (count($recordings) > 0)
    <div class="list-group">
        @foreach ($recordings as $recording)
            <div class="list-group-item list-group-item-action"
                 wire:click="viewRecording({{ $recording->id }})"
```

```
                 wire:click="viewRecording({{ $recording->id }})"
                 style="cursor: pointer;">
                <div class="d-flex align-items-center">
                    {{-- Play Button --}}
                    <button wire:click.stop="playRecording({{ $recording->id }})"
                            class="btn btn-primary btn-sm me-3">
                        <i class="bi bi-play-fill"></i>
                    </button>

                    {{-- Recording Info --}}
                    <div class="flex-grow-1">
                        <div class="d-flex justify-content-between align-items-start">
                            <div>
                                <h6 class="mb-1">{{ $recording->title }}</h6>
                                <small class="text-muted">
                                    {{ $recording->formatted_duration }} •
                                    {{ $recording->human_size }} •
                                    {{ $recording->recorded_at->diffForHumans() }}
                                </small>

                                {{-- Tags --}}
                                @if ($recording->tags && count($recording->tags) > 0)
                                    <div class="mt-1">
                                        @foreach ($recording->tags as $tag)
                                            <span class="badge bg-secondary me-1">{{ $tag }}</span>
                                        @endforeach
                                    </div>
                                @endif
                            </div>

                            {{-- Favorite Icon --}}
                            <button wire:click.stop="toggleFavorite({{ $recording->id }})"
                                    class="btn btn-sm btn-link">
                                <i class="bi bi-star{{ $recording->is_favorite ? '-fill text-warning' : '' }}"></i>
                            </button>
                        </div>
                    </div>
                </div>
            </div>
        @endforeach
    </div>
@else
    <div class="text-center py-5">
        <i class="bi bi-mic" style="font-size: 4rem; color: #ccc;"></i>
        <p class="text-muted mt-3">
            @if ($searchQuery)
                No recordings found matching "{{ $searchQuery }}"
            @else
                No recordings yet
            @endif
        </p>
    </div>
@endif

{{-- Recording Detail Modal --}}
@if ($showRecordingModal && $selectedRecording)
    <div class="modal show d-block" tabindex="-1" style="background: rgba(0,0,0,0.5);">
        <div class="modal-dialog">
            <div class="modal-content">
                <div class="modal-header">
                    <h5 class="modal-title">Recording Details</h5>
                    <button type="button" class="btn-close" wire:click="closeModal"></button>
                </div>
                <div class="modal-body">
                    @if ($editingRecordingId === $selectedRecording->id)
                        {{-- Edit Form --}}
```

```
                <div class="mb-3">
                    <label class="form-label">Title</label>
                    <input type="text" wire:model="editTitle" class="form-control">
                </div>
                <div class="mb-3">
                    <label class="form-label">Notes</label>
                    <textarea wire:model="editNotes" class="form-control" rows="4"></textarea>
                </div>
                <div class="mb-3">
                    <label class="form-label">Tags (comma separated)</label>
                    <input type="text" wire:model="editTags" class="form-control"
                           placeholder="work, important, meeting">
                </div>
            @else
                {{-- View Mode --}}
                <h6>{{ $selectedRecording->title }}</h6>

                @if ($selectedRecording->notes)
                    <p class="text-muted">{{ $selectedRecording->notes }}</p>
                @endif

                {{-- Tags --}}
                @if ($selectedRecording->tags && count($selectedRecording->tags) > 0)
                    <div class="mb-3">
                        @foreach ($selectedRecording->tags as $tag)
                            <span class="badge bg-secondary me-1">{{ $tag }}</span>
                        @endforeach
                    </div>
                @endif

                {{-- Details --}}
                <table class="table table-sm">
                    <tbody>
                        <tr>
                            <td><strong>Duration</strong></td>
                            <td>{{ $selectedRecording->formatted_duration }}</td>
                        </tr>
                        <tr>
                            <td><strong>Size</strong></td>
                            <td>{{ $selectedRecording->human_size }}</td>
                        </tr>
                        <tr>
                            <td><strong>Format</strong></td>
                            <td>{{ strtoupper($selectedRecording->format) }}</td>
                        </tr>
                        <tr>
                            <td><strong>Recorded</strong></td>
                            <td>{{ $selectedRecording->recorded_at->format('M d, Y h:i A') }}</td>
                        </tr>
                    </tbody>
                </table>

                {{-- Play Button --}}
                <div class="d-grid">
                    <button wire:click="playRecording({{ $selectedRecording->id }})"
                            class="btn btn-primary btn-lg">
                        <i class="bi bi-play-fill"></i> Play Recording
                    </button>
                </div>
            @endif
        </div>
        <div class="modal-footer">
            @if ($editingRecordingId === $selectedRecording->id)
                <button wire:click="saveRecordingEdits" class="btn btn-primary">Save</button>
                <button wire:click="cancelEdit" class="btn btn-secondary">Cancel</button>
```

```
                        @else
                            <button wire:click="editRecording({{ $selectedRecording->id }})"
                                    class="btn btn-outline-primary">
                                Edit
                            </button>
                            <button wire:click="deleteRecording({{ $selectedRecording->id }})"
                                    class="btn btn-outline-danger"
                                    onclick="return confirm('Are you sure?')">
                                Delete
                            </button>
                        @endif
                    </div>
                </div>
            </div>
        </div>
    @endif
</div>

{{-- Bootstrap Icons --}}
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-icons.css">

{{-- Recording Timer Script --}}
<script>
    let timerInterval = null;
    let seconds = 0;

    window.addEventListener('start-timer', () => {
        seconds = 0;
        timerInterval = setInterval(() => {
            seconds++;
            updateTimerDisplay();
        }, 1000);
    });

    window.addEventListener('stop-timer', () => {
        if (timerInterval) {
            clearInterval(timerInterval);
            timerInterval = null;
        }
    });

    window.addEventListener('pause-timer', () => {
        if (timerInterval) {
            clearInterval(timerInterval);
            timerInterval = null;
        }
    });

    window.addEventListener('resume-timer', () => {
        timerInterval = setInterval(() => {
            seconds++;
            updateTimerDisplay();
        }, 1000);
    });

    function updateTimerDisplay() {
        const minutes = Math.floor(seconds / 60);
        const secs = seconds % 60;
        const display = `${String(minutes).padStart(2, '0')}:${String(secs).padStart(2, '0')}`;
        const timerElement = document.getElementById('recording-timer');
        if (timerElement) {
            timerElement.textContent = display;
        }
    }
</script>
```

Step 8: Add Route

```
// routes/web.php
use App\Livewire\VoiceRecorder;

Route::get('/recorder', VoiceRecorder::class)->name('recorder');
```

Complete Audio Recording Features:

- ✅ Record audio with device microphone
- ✅ Real-time recording timer
- ✅ Pause and resume recording
- ✅ Save recordings to database
- ✅ Edit recording titles, notes, and tags
- ✅ Search recordings
- ✅ Filter favorites
- ✅ Play recordings with native player
- ✅ Delete recordings
- ✅ Display duration and file size

Let me know if you want me to continue with Part 7 (Scanner), Part 8 (Network), Part 9 (API Integration), and the real-world examples! This is becoming a very comprehensive guide.

Part 7: Scanner Integration - Complete QR/Barcode System

Overview

Build a complete barcode and QR code scanning system with history, product lookup, and inventory management.

Step 1: Install Scanner Plugin

```
composer require nativephp/mobile-scanner
```

Step 2: Enable Scanner Permission

```
// config/nativephp.php
'permissions' => [
    'camera' => true,  // Scanner uses camera
    'scanner' => true,
],
```

Step 3: Create Scans Migration

```
php artisan make:migration create_scans_table
```

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('scans', function (Blueprint $table) {
            $table->id();
            $table->string('code'); // The scanned code value
            $table->string('format'); // qr, ean13, code128, etc.
            $table->string('type')->default('unknown'); // product, url, text, contact, etc.
            $table->text('raw_data'); // Raw scanned data
            $table->json('parsed_data')->nullable(); // Parsed structured data
            $table->string('product_name')->nullable(); // If it's a product
            $table->text('product_description')->nullable();
            $table->decimal('product_price', 10, 2)->nullable();
            $table->string('product_image_url')->nullable();
            $table->integer('scan_count')->default(1); // How many times scanned
            $table->timestamp('last_scanned_at');
            $table->boolean('is_favorite')->default(false);
            $table->json('metadata')->nullable(); // Additional info
            $table->timestamps();
            $table->softDeletes();

            // Indexes
            $table->index('code');
            $table->index('format');
            $table->index('type');
            $table->index('last_scanned_at');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('scans');
    }
};
```

```
php artisan migrate
```

Step 4: Create Scan Model

```
php artisan make:model Scan
```

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Scan extends Model
{
    use SoftDeletes;

    protected $fillable = [
        'code',
        'format',
```

```
        'type',
        'raw_data',
        'parsed_data',
        'product_name',
        'product_description',
        'product_price',
        'product_image_url',
        'scan_count',
        'last_scanned_at',
        'is_favorite',
        'metadata',
    ];

    protected $casts = [
        'parsed_data' => 'array',
        'metadata' => 'array',
        'last_scanned_at' => 'datetime',
        'is_favorite' => 'boolean',
        'product_price' => 'decimal:2',
    ];

    /**
     * Get icon based on scan type
     */
    public function getIconAttribute(): string
    {
        return match($this->type) {
            'product' => 'bi-box-seam',
            'url' => 'bi-link-45deg',
            'text' => 'bi-file-text',
            'contact' => 'bi-person-vcard',
            'wifi' => 'bi-wifi',
            'email' => 'bi-envelope',
            'phone' => 'bi-telephone',
            default => 'bi-qr-code',
        };
    }

    /**
     * Get formatted type
     */
    public function getFormattedTypeAttribute(): string
    {
        return ucfirst($this->type);
    }

    /**
     * Get display text
     */
    public function getDisplayTextAttribute(): string
    {
        if ($this->product_name) {
            return $this->product_name;
        }

        if ($this->type === 'url') {
            return parse_url($this->raw_data, PHP_URL_HOST) ?? $this->raw_data;
        }

        return strlen($this->raw_data) > 50
            ? substr($this->raw_data, 0, 50) . '...'
            : $this->raw_data;
    }

    /**
     * Scope for favorites
```

```
     * Scope for favorites
     */
    public function scopeFavorites($query)
    {
        return $query->where('is_favorite', true);
    }

    /**
     * Scope by type
     */
    public function scopeByType($query, string $type)
    {
        return $query->where('type', $type);
    }

    /**
     * Increment scan count
     */
    public function incrementScanCount(): void
    {
        $this->increment('scan_count');
        $this->update(['last_scanned_at' => now()]);
    }
}
```

Step 5: Create Scanner Service

Create `app/Services/ScannerService.php`:

```
<?php

namespace App\Services;

use App\Models\Scan;
use Illuminate\Support\Facades\Http;

class ScannerService
{
    /**
     * Process scanned code
     */
    public function processScan(string $code, string $format): Scan
    {
        // Check if already scanned
        $existingScan = Scan::where('code', $code)
            ->where('format', $format)
            ->first();

        if ($existingScan) {
            $existingScan->incrementScanCount();
            return $existingScan;
        }

        // Determine type
        $type = $this->determineType($code, $format);

        // Parse data
        $parsedData = $this->parseData($code, $type);

        // Create new scan
        $scan = Scan::create([
            'code' => $code,
            'format' => $format,
            'type' => $type,
            'raw_data' => $code,
```

```
            'parsed_data' => $parsedData,
            'last_scanned_at' => now(),
        ]);

        // If it's a product barcode, try to fetch product info
        if (in_array($format, ['ean13', 'ean8', 'upca', 'upce'])) {
            $this->fetchProductInfo($scan);
        }

        return $scan;
    }

    /**
     * Determine scan type
     */
    private function determineType(string $code, string $format): string
    {
        // URL
        if (filter_var($code, FILTER_VALIDATE_URL)) {
            return 'url';
        }

        // Email
        if (filter_var($code, FILTER_VALIDATE_EMAIL)) {
            return 'email';
        }

        // Phone number
        if (preg_match('/^[\+]?[(]?[0-9]{3}[)]?[-\s\.]?[0-9]{3}[-\s\.]?[0-9]{4,6}$/', $code)) {
            return 'phone';
        }

        // WiFi (format: WIFI:T:WPA;S:NetworkName;P:Password;;)
        if (str_starts_with($code, 'WIFI:')) {
            return 'wifi';
        }

        // Contact (vCard format)
        if (str_starts_with($code, 'BEGIN:VCARD')) {
            return 'contact';
        }

        // Product barcodes
        if (in_array($format, ['ean13', 'ean8', 'upca', 'upce', 'code128'])) {
            return 'product';
        }

        return 'text';
    }

    /**
     * Parse scanned data
     */
    private function parseData(string $code, string $type): ?array
    {
        switch ($type) {
            case 'url':
                return [
                    'url' => $code,
                    'domain' => parse_url($code, PHP_URL_HOST),
                ];

            case 'wifi':
                preg_match('/WIFI:T:([^;]+);S:([^;]+);P:([^;]+);;/', $code, $matches);
                return [
                    'security' => $matches[1] ?? null,
```

```
                'security' => $matches[1] ?? null,
                'ssid' => $matches[2] ?? null,
                'password' => $matches[3] ?? null,
            ];

        case 'contact':
            // Parse vCard
            $lines = explode("\n", $code);
            $data = [];
            foreach ($lines as $line) {
                if (str_contains($line, ':')) {
                    [$key, $value] = explode(':', $line, 2);
                    $data[trim($key)] = trim($value);
                }
            }
            return $data;

        default:
            return null;
    }
}

/**
 * Fetch product information from API
 */
private function fetchProductInfo(Scan $scan): void
{
    try {
        // Using Open Food Facts API (free, no key required)
        $response = Http::get("https://world.openfoodfacts.org/api/v0/product/{$scan->code}.json");

        if ($response->successful()) {
            $data = $response->json();

            if ($data['status'] == 1 && isset($data['product'])) {
                $product = $data['product'];

                $scan->update([
                    'product_name' => $product['product_name'] ?? null,
                    'product_description' => $product['generic_name'] ?? null,
                    'product_image_url' => $product['image_url'] ?? null,
                    'metadata' => [
                        'brand' => $product['brands'] ?? null,
                        'categories' => $product['categories'] ?? null,
                        'quantity' => $product['quantity'] ?? null,
                        'nutrition_grade' => $product['nutrition_grade_fr'] ?? null,
                    ],
                ]);
            }
        }
    } catch (\Exception $e) {
        // Silently fail - product info is optional
        logger()->error('Failed to fetch product info: ' . $e->getMessage());
    }
}

/**
 * Update scan
 */
public function updateScan(Scan $scan, array $data): Scan
{
    $scan->update($data);
    return $scan->fresh();
}

/**
```

```
     * Delete scan
     */
    public function deleteScan(Scan $scan): bool
    {
        return $scan->delete();
    }

    /**
     * Toggle favorite
     */
    public function toggleFavorite(Scan $scan): Scan
    {
        $scan->update(['is_favorite' => !$scan->is_favorite]);
        return $scan->fresh();
    }

    /**
     * Get all scans
     */
    public function getAllScans()
    {
        return Scan::latest('last_scanned_at')->get();
    }

    /**
     * Get scans by type
     */
    public function getScansByType(string $type)
    {
        return Scan::byType($type)->latest('last_scanned_at')->get();
    }

    /**
     * Get favorites
     */
    public function getFavorites()
    {
        return Scan::favorites()->latest('last_scanned_at')->get();
    }

    /**
     * Search scans
     */
    public function searchScans(string $query)
    {
        return Scan::where('code', 'like', "%{$query}%")
            ->orWhere('raw_data', 'like', "%{$query}%")
            ->orWhere('product_name', 'like', "%{$query}%")
            ->latest('last_scanned_at')
            ->get();
    }

    /**
     * Get scan statistics
     */
    public function getStatistics(): array
    {
        return [
            'total_scans' => Scan::count(),
            'total_unique_codes' => Scan::distinct('code')->count(),
            'most_scanned' => Scan::orderBy('scan_count', 'desc')->first(),
            'by_type' => Scan::selectRaw('type, COUNT(*) as count')
                ->groupBy('type')
                ->get()
                ->pluck('count', 'type')
```

```
                ->toArray(),
        ];
    }
}
```

Step 6: Create Scanner Component

```
php artisan make:livewire Scanner/BarcodeScanner
```

```
<?php

namespace App\Livewire\Scanner;

use Livewire\Component;
use Native\Mobile\Facades\Scanner;
use Native\Mobile\Events\Scanner\CodeScanned;
use Native\Mobile\Attributes\OnNative;
use App\Services\ScannerService;
use App\Models\Scan;

class BarcodeScanner extends Component
{
    public $scans = [];
    public $isScanning = false;
    public $currentFilter = 'all';
    public $searchQuery = '';
    public $selectedScan = null;
    public $showScanModal = false;
    public $statistics = [];

    // Edit scan
    public $editingScanId = null;
    public $editProductName = '';
    public $editProductPrice = '';
    public $editNotes = '';

    // Scanner settings
    public $continuousMode = false;
    public $scanFormats = ['all']; // Default to all formats

    protected $scannerService;

    public function boot(ScannerService $scannerService)
    {
        $this->scannerService = $scannerService;
    }

    public function mount()
    {
        $this->loadScans();
        $this->loadStatistics();
    }

    /**
     * Load scans
     */
    public function loadScans()
    {
        if ($this->currentFilter === 'all') {
            $this->scans = $this->scannerService->getAllScans();
        } elseif ($this->currentFilter === 'favorites') {
            $this->scans = $this->scannerService->getFavorites();
        } elseif ($this->searchQuery) {
            $this->scans = $this->scannerService->searchScans($this->searchQuery);
```

```
        $this->scans = $this->scannerService->searchScans($this->searchQuery);
    } else {
        $this->scans = $this->scannerService->getScansByType($this->currentFilter);
    }
}

/**
 * Load statistics
 */
public function loadStatistics()
{
    $this->statistics = $this->scannerService->getStatistics();
}

/**
 * Start scanning
 */
public function startScanning()
{
    try {
        $scanner = Scanner::scan()
            ->prompt('Point camera at barcode or QR code')
            ->continuous($this->continuousMode);

        // Set formats if not 'all'
        if ($this->scanFormats !== ['all']) {
            $scanner->formats($this->scanFormats);
        }

        $scanner->id('barcode-scan-' . time());

        $this->isScanning = true;
        $this->dispatch('scanning-started');

    } catch (\Exception $e) {
        session()->flash('error', 'Failed to start scanner: ' . $e->getMessage());
    }
}

/**
 * Stop scanning
 */
public function stopScanning()
{
    Scanner::stop();
    $this->isScanning = false;
    $this->dispatch('scanning-stopped');
}

/**
 * Handle code scanned
 */
#[OnNative(CodeScanned::class)]
public function handleCodeScanned($data, $format, $id = null)
{
    try {
        // Process the scan
        $scan = $this->scannerService->processScan($data, $format);

        // Reload scans and statistics
        $this->loadScans();
        $this->loadStatistics();

        // Show success notification
        $this->dispatch('scan-success', message: 'Code scanned successfully!');
```

```
            // If not in continuous mode, stop scanning and show result
            if (!$this->continuousMode) {
                $this->stopScanning();
                $this->viewScan($scan->id);
            }

        } catch (\Exception $e) {
            session()->flash('error', 'Failed to process scan: ' . $e->getMessage());
        }
    }

    /**
     * Change filter
     */
    public function setFilter($filter)
    {
        $this->currentFilter = $filter;
        $this->searchQuery = '';
        $this->loadScans();
    }

    /**
     * Search scans
     */
    public function updatedSearchQuery()
    {
        $this->loadScans();
    }

    /**
     * Toggle continuous mode
     */
    public function toggleContinuousMode()
    {
        $this->continuousMode = !$this->continuousMode;
    }

    /**
     * View scan details
     */
    public function viewScan($scanId)
    {
        $this->selectedScan = Scan::find($scanId);
        $this->showScanModal = true;
    }

    /**
     * Close modal
     */
    public function closeModal()
    {
        $this->showScanModal = false;
        $this->selectedScan = null;
        $this->cancelEdit();
    }

    /**
     * Toggle favorite
     */
    public function toggleFavorite($scanId)
    {
        $scan = Scan::find($scanId);
        if ($scan) {
            $this->scannerService->toggleFavorite($scan);
            $this->loadScans();
```

```
        // Update selected scan if it's open
        if ($this->selectedScan && $this->selectedScan->id === $scanId) {
            $this->selectedScan = $scan->fresh();
        }
    }
}

/**
 * Start editing
 */
public function editScan($scanId)
{
    $scan = Scan::find($scanId);
    if ($scan) {
        $this->editingScanId = $scan->id;
        $this->editProductName = $scan->product_name ?? '';
        $this->editProductPrice = $scan->product_price ?? '';
        $this->editNotes = $scan->metadata['notes'] ?? '';
    }
}

/**
 * Save edits
 */
public function saveScanEdits()
{
    $this->validate([
        'editProductName' => 'nullable|string|max:255',
        'editProductPrice' => 'nullable|numeric|min:0',
        'editNotes' => 'nullable|string|max:1000',
    ]);

    $scan = Scan::find($this->editingScanId);
    if ($scan) {
        $metadata = $scan->metadata ?? [];
        $metadata['notes'] = $this->editNotes;

        $this->scannerService->updateScan($scan, [
            'product_name' => $this->editProductName ?: null,
            'product_price' => $this->editProductPrice ?: null,
            'metadata' => $metadata,
        ]);

        $this->loadScans();
        $this->selectedScan = $scan->fresh();
        $this->cancelEdit();

        session()->flash('success', 'Scan updated successfully!');
    }
}

/**
 * Cancel editing
 */
public function cancelEdit()
{
    $this->editingScanId = null;
    $this->editProductName = '';
    $this->editProductPrice = '';
    $this->editNotes = '';
}

/**
 * Delete scan
 */
```

```
    public function deleteScan($scanId)
    {
        $scan = Scan::find($scanId);
        if ($scan) {
            $this->scannerService->deleteScan($scan);
            $this->loadScans();
            $this->loadStatistics();
            $this->closeModal();

            session()->flash('success', 'Scan deleted successfully!');
        }
    }

    /**
     * Open URL in browser
     */
    public function openUrl($scanId)
    {
        $scan = Scan::find($scanId);
        if ($scan && $scan->type === 'url') {
            $this->dispatch('open-url', url: $scan->raw_data);
        }
    }

    /**
     * Copy to clipboard
     */
    public function copyToClipboard($scanId)
    {
        $scan = Scan::find($scanId);
        if ($scan) {
            $this->dispatch('copy-to-clipboard', text: $scan->raw_data);
            session()->flash('success', 'Copied to clipboard!');
        }
    }

    public function render()
    {
        return view('livewire.scanner.barcode-scanner');
    }
}
```

Step 7: Create Scanner View

Create `resources/views/livewire/scanner/barcode-scanner.blade.php`:

```
<div class="container py-4">
    {{-- Flash Messages --}}
    @if (session()->has('success'))
        <div class="alert alert-success alert-dismissible fade show">
            {{ session('success') }}
            <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
        </div>
    @endif

    @if (session()->has('error'))
        <div class="alert alert-danger alert-dismissible fade show">
            {{ session('error') }}
            <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
        </div>
    @endif

    {{-- Statistics Card --}}
    <div class="card mb-4">
```

```
    <div class="card-body">
        <div class="row text-center">
            <div class="col-4">
                <h4 class="mb-0">{{ $statistics['total_scans'] ?? 0 }}</h4>
                <small class="text-muted">Total Scans</small>
            </div>
            <div class="col-4">
                <h4 class="mb-0">{{ $statistics['total_unique_codes'] ?? 0 }}</h4>
                <small class="text-muted">Unique Codes</small>
            </div>
            <div class="col-4">
                @if (isset($statistics['most_scanned']) && $statistics['most_scanned'])
                    <h4 class="mb-0">{{ $statistics['most_scanned']->scan_count }}</h4>
                    <small class="text-muted">Most Scanned</small>
                @else
                    <h4 class="mb-0">-</h4>
                    <small class="text-muted">Most Scanned</small>
                @endif
            </div>
        </div>
    </div>
</div>

{{-- Scanner Controls --}}
<div class="card mb-4">
    <div class="card-body text-center">
        @if ($isScanning)
            {{-- Scanning in Progress --}}
            <div class="mb-3">
                <div class="spinner-border text-primary" role="status">
                    <span class="visually-hidden">Scanning...</span>
                </div>
            </div>
            <p class="mb-3">Point camera at barcode or QR code</p>
            <button wire:click="stopScanning" class="btn btn-danger btn-lg">
                <i class="bi bi-stop-circle"></i> Stop Scanning
            </button>
        @else
            {{-- Start Scanning --}}
            <button wire:click="startScanning" class="btn btn-primary btn-lg mb-3">
                <i class="bi bi-upc-scan"></i> Start Scanner
            </button>

            {{-- Continuous Mode Toggle --}}
            <div class="form-check form-switch d-flex justify-content-center">
                <input class="form-check-input me-2"
                       type="checkbox"
                       wire:model.live="continuousMode"
                       id="continuousMode">
                <label class="form-check-label" for="continuousMode">
                    Continuous Mode
                </label>
            </div>
        @endif
    </div>
</div>

{{-- Search Bar --}}
<div class="input-group mb-3">
    <input type="text"
           wire:model.live="searchQuery"
           class="form-control"
           placeholder="Search scans...">
    @if ($searchQuery)
        <button wire:click="$set('searchQuery', '')" class="btn btn-outline-secondary">
            <i class="bi bi-x"></i>
```

```
        </button>
    @endif
</div>

{{-- Filter Tabs --}}
<ul class="nav nav-pills mb-4" style="overflow-x: auto; white-space: nowrap;">
    <li class="nav-item">
        <a class="nav-link {{ $currentFilter === 'all' ? 'active' : '' }}"
           wire:click="setFilter('all')"
           href="#">All</a>
    </li>
    <li class="nav-item">
        <a class="nav-link {{ $currentFilter === 'favorites' ? 'active' : '' }}"
           wire:click="setFilter('favorites')"
           href="#">Favorites</a>
    </li>
    <li class="nav-item">
        <a class="nav-link {{ $currentFilter === 'product' ? 'active' : '' }}"
           wire:click="setFilter('product')"
           href="#">Products</a>
    </li>
    <li class="nav-item">
        <a class="nav-link {{ $currentFilter === 'url' ? 'active' : '' }}"
           wire:click="setFilter('url')"
           href="#">URLs</a>
    </li>
    <li class="nav-item">
        <a class="nav-link {{ $currentFilter === 'text' ? 'active' : '' }}"
           wire:click="setFilter('text')"
           href="#">Text</a>
    </li>
</ul>

{{-- Scans List --}}
@if (count($scans) > 0)
    <div class="list-group">
        @foreach ($scans as $scan)
            <div class="list-group-item list-group-item-action"
                 wire:click="viewScan({{ $scan->id }})"
                 style="cursor: pointer;">
                <div class="d-flex align-items-center">
                    {{-- Icon --}}
                    <div class="me-3">
                        <i class="{{ $scan->icon }}" style="font-size: 2rem; color: #6c757d;"></i>
                    </div>

                    {{-- Scan Info --}}
                    <div class="flex-grow-1">
                        <div class="d-flex justify-content-between align-items-start">
                            <div>
                                <h6 class="mb-1">{{ $scan->display_text }}</h6>
                                <small class="text-muted">
                                    <span class="badge bg-secondary me-1">{{ strtoupper($scan->format) }}</span>
                                    <span class="badge bg-info me-1">{{ $scan->formatted_type }}</span>
                                    @if ($scan->scan_count > 1)
                                        <span class="badge bg-success">Scanned {{ $scan->scan_count }}x</span>
                                    @endif
                                </small>
                                <br>
                                <small class="text-muted">{{ $scan->last_scanned_at->diffForHumans() }}</small>
                            </div>

                            {{-- Favorite Icon --}}
                            <button wire:click.stop="toggleFavorite({{ $scan->id }})"
                                    class="btn btn-sm btn-link">
```

```
                                <i class="bi bi-star{{ $scan->is_favorite ? '-fill text-warning' : '' }}"></i>
                            </button>
                        </div>
                    </div>
                </div>
            </div>
        @endforeach
    </div>
@else
    <div class="text-center py-5">
        <i class="bi bi-qr-code-scan" style="font-size: 4rem; color: #ccc;"></i>
        <p class="text-muted mt-3">
            @if ($searchQuery)
                No scans found matching "{{ $searchQuery }}"
            @else
                No scans yet. Start scanning!
            @endif
        </p>
    </div>
@endif

{{-- Scan Detail Modal --}}
@if ($showScanModal && $selectedScan)
    <div class="modal show d-block" tabindex="-1" style="background: rgba(0,0,0,0.5);">
        <div class="modal-dialog modal-dialog-scrollable">
            <div class="modal-content">
                <div class="modal-header">
                    <h5 class="modal-title">
                        <i class="{{ $selectedScan->icon }} me-2"></i>
                        {{ $selectedScan->formatted_type }}
                    </h5>
                    <button type="button" class="btn-close" wire:click="closeModal"></button>
                </div>
                <div class="modal-body">
                    @if ($editingScanId === $selectedScan->id)
                        {{-- Edit Form --}}
                        <div class="mb-3">
                            <label class="form-label">Product Name</label>
                            <input type="text" wire:model="editProductName" class="form-control">
                        </div>
                        <div class="mb-3">
                            <label class="form-label">Price</label>
                            <input type="number" step="0.01" wire:model="editProductPrice" class="form-control">
                        </div>
                        <div class="mb-3">
                            <label class="form-label">Notes</label>
                            <textarea wire:model="editNotes" class="form-control" rows="3"></textarea>
                        </div>
                    @else
                        {{-- View Mode --}}

                        {{-- Product Info (if available) --}}
                        @if ($selectedScan->product_name)
                            <div class="card mb-3">
                                <div class="card-body">
                                    @if ($selectedScan->product_image_url)
                                        <img src="{{ $selectedScan->product_image_url }}"
                                             class="img-fluid rounded mb-3"
                                             alt="{{ $selectedScan->product_name }}">
                                    @endif

                                    <h6>{{ $selectedScan->product_name }}</h6>

                                    @if ($selectedScan->product_description)
                                        <p class="text-muted small">{{ $selectedScan->product_description }}</p>
                                    @endif
```

```
                @endif

                @if ($selectedScan->product_price)
                    <p class="mb-0"><strong>Price:</strong> ${{ number_format($selectedScan->product_price,
                @endif

                @if ($selectedScan->metadata)
                    @if (isset($selectedScan->metadata['brand']))
                        <p class="mb-0"><strong>Brand:</strong> {{ $selectedScan->metadata['brand'] }}</p>
                    @endif
                    @if (isset($selectedScan->metadata['nutrition_grade']))
                        <p class="mb-0">
                            <strong>Nutrition Grade:</strong>
                            <span class="badge bg-{{ $selectedScan->metadata['nutrition_grade'] === 'a' ? 's
                                {{ strtoupper($selectedScan->metadata['nutrition_grade']) }}
                            </span>
                        </p>
                    @endif
                @endif
            </div>
        </div>
    @endif

    {{-- Raw Data --}}
    <div class="card mb-3">
        <div class="card-body">
            <h6 class="card-subtitle mb-2 text-muted">Scanned Data</h6>
            <p class="card-text font-monospace small">{{ $selectedScan->raw_data }}</p>
            <button wire:click="copyToClipboard({{ $selectedScan->id }})"
                    class="btn btn-sm btn-outline-secondary">
                <i class="bi bi-clipboard"></i> Copy
            </button>
        </div>
    </div>

    {{-- Scan Details --}}
    <table class="table table-sm">
        <tbody>
            <tr>
                <td><strong>Code</strong></td>
                <td>{{ $selectedScan->code }}</td>
            </tr>
            <tr>
                <td><strong>Format</strong></td>
                <td><span class="badge bg-secondary">{{ strtoupper($selectedScan->format) }}</span></td>
            </tr>
            <tr>
                <td><strong>Type</strong></td>
                <td><span class="badge bg-info">{{ $selectedScan->formatted_type }}</span></td>
            </tr>
            <tr>
                <td><strong>Scan Count</strong></td>
                <td>{{ $selectedScan->scan_count }}</td>
            </tr>
            <tr>
                <td><strong>First Scanned</strong></td>
                <td>{{ $selectedScan->created_at->format('M d, Y h:i A') }}</td>
            </tr>
            <tr>
                <td><strong>Last Scanned</strong></td>
                <td>{{ $selectedScan->last_scanned_at->format('M d, Y h:i A') }}</td>
            </tr>
        </tbody>
    </table>

    {{-- Notes --}}
```

```
                            @if ($selectedScan->metadata && isset($selectedScan->metadata['notes']))
                                <div class="alert alert-info">
                                    <strong>Notes:</strong><br>
                                    {{ $selectedScan->metadata['notes'] }}
                                </div>
                            @endif

                            {{-- Action Buttons --}}
                            @if ($selectedScan->type === 'url')
                                <div class="d-grid gap-2 mb-3">
                                    <button wire:click="openUrl({{ $selectedScan->id }})" class="btn btn-primary">
                                        <i class="bi bi-box-arrow-up-right"></i> Open URL
                                    </button>
                                </div>
                            @endif
                        @endif
                    </div>
                    <div class="modal-footer">
                        @if ($editingScanId === $selectedScan->id)
                            <button wire:click="saveScanEdits" class="btn btn-primary">Save</button>
                            <button wire:click="cancelEdit" class="btn btn-secondary">Cancel</button>
                        @else
                            <button wire:click="editScan({{ $selectedScan->id }})" class="btn btn-outline-primary">
                                Edit
                            </button>
                            <button wire:click="deleteScan({{ $selectedScan->id }})"
                                    class="btn btn-outline-danger"
                                    onclick="return confirm('Are you sure?')">
                                Delete
                            </button>
                        @endif
                    </div>
                </div>
            </div>
        </div>
    @endif
</div>

{{-- Bootstrap Icons --}}
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-icons.css">

{{-- Scripts for handling events --}}
<script>
    // Handle scan success notification
    window.addEventListener('scan-success', (event) => {
        // You can add custom notification here
        console.log(event.detail.message);
    });

    // Handle open URL
    window.addEventListener('open-url', (event) => {
        window.open(event.detail.url, '_blank');
    });

    // Handle copy to clipboard
    window.addEventListener('copy-to-clipboard', (event) => {
        navigator.clipboard.writeText(event.detail.text);
    });
</script>
```

Step 8: Add Route

```
// routes/web.php
use App\Livewire\Scanner\BarcodeScanner;

Route::get('/scanner', BarcodeScanner::class)->name('scanner');
```

Complete Scanner Features:

- ✅ Scan QR codes and barcodes (EAN13, EAN8, UPC, Code128, etc.)
- ✅ Automatic product information lookup via Open Food Facts API
- ✅ Auto-detect scan type (URL, email, phone, WiFi, contact, product, text)
- ✅ Continuous scanning mode
- ✅ Scan history with count tracking
- ✅ Search and filter by type
- ✅ Favorites system
- ✅ Edit product names, prices, and notes
- ✅ Copy scanned data to clipboard
- ✅ Open URLs directly from app
- ✅ Statistics dashboard
- ✅ Product images and nutrition grades

I'll continue with the remaining parts. Should I keep going with Part 8 (Network & Connectivity), Part 9 (Complete API Integration), and the Real-World Examples?

Part 8: Network & Connectivity Management

Overview

Build a robust network monitoring system with offline mode, sync queue, and connectivity indicators.

Step 1: Install Network Plugin

```
composer require nativephp/mobile-network
```

Step 2: Enable Permission

```
// config/nativephp.php
'permissions' => [
    'network_state' => true,
],
```

Step 3: Create Network Service

Create `app/Services/NetworkService.php`:

```
<?php

namespace App\Services;

use Native\Mobile\Facades\Network;
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Facades\Log;

class NetworkService
{
    /**
     * Check if device is connected
     */
    public function isConnected(): bool
    {
        try {
            return Network::isConnected();
```

```
        } catch (\Exception $e) {
            Log::error('Failed to check network connection: ' . $e->getMessage());
            return false;
        }
    }

    /**
     * Get connection type
     */
    public function getConnectionType(): string
    {
        try {
            return Network::getConnectionType(); // 'wifi', 'cellular', 'none'
        } catch (\Exception $e) {
            Log::error('Failed to get connection type: ' . $e->getMessage());
            return 'none';
        }
    }

    /**
     * Get connection status with details
     */
    public function getStatus(): array
    {
        $isConnected = $this->isConnected();
        $type = $this->getConnectionType();

        return [
            'is_connected' => $isConnected,
            'type' => $type,
            'quality' => $this->getConnectionQuality($type),
            'status_text' => $this->getStatusText($isConnected, $type),
            'icon' => $this->getStatusIcon($type),
            'color' => $this->getStatusColor($isConnected, $type),
        ];
    }

    /**
     * Get connection quality estimate
     */
    private function getConnectionQuality(string $type): string
    {
        return match($type) {
            'wifi' => 'excellent',
            'cellular' => 'good',
            default => 'offline',
        };
    }

    /**
     * Get human-readable status text
     */
    private function getStatusText(bool $isConnected, string $type): string
    {
        if (!$isConnected) {
            return 'Offline - No internet connection';
        }

        return match($type) {
            'wifi' => 'Connected via WiFi',
            'cellular' => 'Connected via Mobile Data',
            default => 'Connected',
        };
    }

    ...
```

```
    /**
     * Get status icon
     */
    private function getStatusIcon(string $type): string
    {
        return match($type) {
            'wifi' => 'bi-wifi',
            'cellular' => 'bi-signal',
            default => 'bi-wifi-off',
        };
    }

    /**
     * Get status color
     */
    private function getStatusColor(bool $isConnected, string $type): string
    {
        if (!$isConnected) {
            return 'danger';
        }

        return match($type) {
            'wifi' => 'success',
            'cellular' => 'info',
            default => 'secondary',
        };
    }

    /**
     * Store network status in cache
     */
    public function cacheStatus(): void
    {
        $status = $this->getStatus();
        Cache::put('network_status', $status, now()->addMinutes(5));
    }

    /**
     * Get cached network status
     */
    public function getCachedStatus(): ?array
    {
        return Cache::get('network_status');
    }
}
```

Step 4: Create Sync Queue Migration

```
php artisan make:migration create_sync_queue_table
```

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('sync_queue', function (Blueprint $table) {
            $table->id();
            $table->string('model_type'); // Photo, AudioRecording, etc.
            $table->unsignedBigInteger('model_id');
            $table->string('action'); // create, update, delete
            $table->json('data')->nullable(); // The data to sync
            $table->integer('retry_count')->default(0);
            $table->timestamp('last_attempt_at')->nullable();
            $table->string('error_message')->nullable();
            $table->string('status')->default('pending'); // pending, syncing, synced, failed
            $table->timestamps();

            $table->index(['model_type', 'model_id']);
            $table->index('status');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('sync_queue');
    }
};
```

```
php artisan migrate
```

Step 5: Create SyncQueue Model

```
php artisan make:model SyncQueue
```

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class SyncQueue extends Model
{
    protected $table = 'sync_queue';

    protected $fillable = [
        'model_type',
        'model_id',
        'action',
        'data',
        'retry_count',
        'last_attempt_at',
        'error_message',
        'status',
    ];

    protected $casts = [
        'data' => 'array',
```

```
        'last_attempt_at' => 'datetime',
    ];

    /**
     * Scope for pending items
     */
    public function scopePending($query)
    {
        return $query->where('status', 'pending');
    }

    /**
     * Scope for failed items
     */
    public function scopeFailed($query)
    {
        return $query->where('status', 'failed');
    }

    /**
     * Mark as syncing
     */
    public function markAsSyncing(): void
    {
        $this->update([
            'status' => 'syncing',
            'last_attempt_at' => now(),
        ]);
    }

    /**
     * Mark as synced
     */
    public function markAsSynced(): void
    {
        $this->update(['status' => 'synced']);
    }

    /**
     * Mark as failed
     */
    public function markAsFailed(string $errorMessage): void
    {
        $this->update([
            'status' => 'failed',
            'error_message' => $errorMessage,
            'retry_count' => $this->retry_count + 1,
        ]);
    }
}
```

Step 6: Create Sync Service

Create `app/Services/SyncService.php`:

```
<?php

namespace App\Services;

use App\Models\SyncQueue;
use App\Models\Photo;
use App\Models\AudioRecording;
use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Log;
```

```php
class SyncService
{
    private $apiService;
    private $networkService;

    public function __construct(ApiService $apiService, NetworkService $networkService)
    {
        $this->apiService = $apiService;
        $this->networkService = $networkService;
    }

    /**
     * Add item to sync queue
     */
    public function queueForSync(string $modelType, int $modelId, string $action, ?array $data = null): void
    {
        SyncQueue::create([
            'model_type' => $modelType,
            'model_id' => $modelId,
            'action' => $action,
            'data' => $data,
            'status' => 'pending',
        ]);
    }

    /**
     * Process sync queue
     */
    public function processSyncQueue(): array
    {
        // Check network connectivity
        if (!$this->networkService->isConnected()) {
            return [
                'success' => false,
                'message' => 'No internet connection',
                'synced' => 0,
                'failed' => 0,
            ];
        }

        $pendingItems = SyncQueue::pending()
            ->where('retry_count', '<', 3) // Max 3 retries
            ->limit(10) // Process 10 at a time
            ->get();

        $syncedCount = 0;
        $failedCount = 0;

        foreach ($pendingItems as $item) {
            $item->markAsSyncing();

            try {
                $this->syncItem($item);
                $item->markAsSynced();
                $syncedCount++;
            } catch (\Exception $e) {
                $item->markAsFailed($e->getMessage());
                $failedCount++;
                Log::error("Sync failed for {$item->model_type}:{$item->model_id} - " . $e->getMessage());
            }
        }

        return [
            'success' => true,
            'synced' => $syncedCount,
```

```
            'failed' => $failedCount,
            'remaining' => SyncQueue::pending()->count(),
        ];
    }

    /**
     * Sync individual item
     */
    private function syncItem(SyncQueue $item): void
    {
        $model = $this->getModel($item->model_type, $item->model_id);

        if (!$model) {
            throw new \Exception('Model not found');
        }

        switch ($item->action) {
            case 'create':
                $this->syncCreate($model, $item->model_type);
                break;
            case 'update':
                $this->syncUpdate($model, $item->model_type);
                break;
            case 'delete':
                $this->syncDelete($model, $item->model_type);
                break;
        }
    }

    /**
     * Get model instance
     */
    private function getModel(string $modelType, int $modelId)
    {
        return match($modelType) {
            'Photo' => Photo::find($modelId),
            'AudioRecording' => AudioRecording::find($modelId),
            default => null,
        };
    }

    /**
     * Sync create action
     */
    private function syncCreate($model, string $modelType): void
    {
        $endpoint = $this->getEndpoint($modelType);
        $data = $this->prepareData($model, $modelType);

        $response = $this->apiService->post($endpoint, $data);

        // Update local model with remote ID
        if (isset($response['id'])) {
            $model->update([
                'is_synced' => true,
                'remote_id' => $response['id'],
            ]);
        }
    }

    /**
     * Sync update action
     */
    private function syncUpdate($model, string $modelType): void
    {
        $endpoint = $this->getEndpoint($modelType) . '/' . $model->remote_id;
```

```
        $endpoint = $this->getEndpoint($modelType) . '/' . $model->remote_id;
        $data = $this->prepareData($model, $modelType);

        $this->apiService->put($endpoint, $data);

        $model->update(['is_synced' => true]);
    }

    /**
     * Sync delete action
     */
    private function syncDelete($model, string $modelType): void
    {
        if ($model->remote_id) {
            $endpoint = $this->getEndpoint($modelType) . '/' . $model->remote_id;
            $this->apiService->delete($endpoint);
        }
    }

    /**
     * Get API endpoint for model type
     */
    private function getEndpoint(string $modelType): string
    {
        return match($modelType) {
            'Photo' => '/api/photos',
            'AudioRecording' => '/api/recordings',
            default => '/api/items',
        };
    }

    /**
     * Prepare data for API
     */
    private function prepareData($model, string $modelType): array
    {
        return match($modelType) {
            'Photo' => [
                'title' => $model->title,
                'description' => $model->description,
                'filename' => $model->filename,
                'size' => $model->size,
                'taken_at' => $model->taken_at->toIso8601String(),
                // Note: For file upload, you'd need to handle multipart form data
            ],
            'AudioRecording' => [
                'title' => $model->title,
                'notes' => $model->notes,
                'duration' => $model->duration,
                'recorded_at' => $model->recorded_at->toIso8601String(),
            ],
            default => [],
        };
    }

    /**
     * Get sync statistics
     */
    public function getStatistics(): array
    {
        return [
            'pending' => SyncQueue::pending()->count(),
            'synced' => SyncQueue::where('status', 'synced')->count(),
            'failed' => SyncQueue::failed()->count(),
            'last_sync' => SyncQueue::where('status', 'synced')
                ->latest('updated_at')
```

```
                ->first()
                ?->updated_at,
        ];
    }

    /**
     * Clear synced items from queue
     */
    public function clearSynced(): int
    {
        return SyncQueue::where('status', 'synced')->delete();
    }

    /**
     * Retry failed items
     */
    public function retryFailed(): void
    {
        SyncQueue::failed()->update([
            'status' => 'pending',
            'error_message' => null,
        ]);
    }
}
```

Step 7: Create Network Status Component

```
php artisan make:livewire NetworkStatus
```

```
<?php

namespace App\Livewire;

use Livewire\Component;
use Native\Mobile\Events\Network\NetworkStatusChanged;
use Native\Mobile\Attributes\OnNative;
use App\Services\NetworkService;
use App\Services\SyncService;

class NetworkStatus extends Component
{
    public $status = [];
    public $syncStats = [];
    public $isSyncing = false;
    public $lastSyncMessage = '';

    protected $networkService;
    protected $syncService;

    public function boot(NetworkService $networkService, SyncService $syncService)
    {
        $this->networkService = $networkService;
        $this->syncService = $syncService;
    }

    public function mount()
    {
        $this->loadStatus();
        $this->loadSyncStats();
    }

    /**
     * Load network status
```

```
     */
    public function loadStatus()
    {
        $this->status = $this->networkService->getStatus();
    }

    /**
     * Load sync statistics
     */
    public function loadSyncStats()
    {
        $this->syncStats = $this->syncService->getStatistics();
    }

    /**
     * Handle network status change
     */
    #[OnNative(NetworkStatusChanged::class)]
    public function handleNetworkChange($isConnected, $connectionType)
    {
        // Update status
        $this->loadStatus();

        // If connected, auto-sync
        if ($isConnected) {
            $this->lastSyncMessage = 'Connected! Auto-syncing...';
            $this->syncNow();
        } else {
            $this->lastSyncMessage = 'Connection lost. Will sync when reconnected.';
        }

        // Flash notification
        session()->flash('network_status', $this->status['status_text']);
    }

    /**
     * Manually trigger sync
     */
    public function syncNow()
    {
        if (!$this->status['is_connected']) {
            session()->flash('error', 'Cannot sync: No internet connection');
            return;
        }

        $this->isSyncing = true;

        try {
            $result = $this->syncService->processSyncQueue();

            $this->loadSyncStats();
            $this->isSyncing = false;

            if ($result['synced'] > 0) {
                $this->lastSyncMessage = "Synced {$result['synced']} items successfully!";
            } else {
                $this->lastSyncMessage = "Everything is up to date.";
            }

            if ($result['failed'] > 0) {
                $this->lastSyncMessage .= " {$result['failed']} failed.";
            }

        } catch (\Exception $e) {
            $this->isSyncing = false;
            $this->lastSyncMessage = "Sync failed: " . $e->getMessage();
```

```
        }
    }

    /**
     * Retry failed syncs
     */
    public function retryFailed()
    {
        $this->syncService->retryFailed();
        $this->loadSyncStats();
        $this->syncNow();
    }

    /**
     * Clear synced items
     */
    public function clearSynced()
    {
        $count = $this->syncService->clearSynced();
        $this->loadSyncStats();
        session()->flash('success', "Cleared {$count} synced items");
    }

    public function render()
    {
        return view('livewire.network-status');
    }
}
```

Step 8: Create Network Status View

Create `resources/views/livewire/network-status.blade.php` :

```
<div class="container py-4">
    {{-- Flash Messages --}}
    @if (session()->has('network_status'))
        <div class="alert alert-info alert-dismissible fade show">
            <i class="{{ $status['icon'] ?? 'bi-info-circle' }}"></i>
            {{ session('network_status') }}
            <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
        </div>
    @endif

    @if (session()->has('success'))
        <div class="alert alert-success alert-dismissible fade show">
            {{ session('success') }}
            <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
        </div>
    @endif

    @if (session()->has('error'))
        <div class="alert alert-danger alert-dismissible fade show">
            {{ session('error') }}
            <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
        </div>
    @endif

    {{-- Current Network Status --}}
    <div class="card mb-4">
        <div class="card-header bg-{{ $status['color'] ?? 'secondary' }} text-white">
            <h5 class="mb-0">
                <i class="{{ $status['icon'] ?? 'bi-wifi-off' }}"></i>
                Network Status
            </h5>
```

```
        </div>
        <div class="card-body">
            <div class="d-flex justify-content-between align-items-center mb-3">
                <div>
                    <h6 class="mb-1">{{ $status['status_text'] ?? 'Unknown' }}</h6>
                    <small class="text-muted">
                        Connection Quality:
                        <span class="badge bg-{{ $status['quality'] === 'excellent' ? 'success' : ($status['quality'] === 'good' ? '
                            {{ ucfirst($status['quality'] ?? 'Unknown') }}
                        </span>
                    </small>
                </div>
                <button wire:click="loadStatus" class="btn btn-sm btn-outline-secondary">
                    <i class="bi bi-arrow-clockwise"></i> Refresh
                </button>
            </div>

            @if ($lastSyncMessage)
                <div class="alert alert-light mb-0">
                    <small>{{ $lastSyncMessage }}</small>
                </div>
            @endif
        </div>
    </div>

    {{-- Sync Queue Status --}}
    <div class="card mb-4">
        <div class="card-header">
            <h5 class="mb-0">
                <i class="bi bi-cloud-arrow-up"></i>
                Sync Status
            </h5>
        </div>
        <div class="card-body">
            <div class="row text-center mb-3">
                <div class="col-4">
                    <h4 class="mb-0 text-warning">{{ $syncStats['pending'] ?? 0 }}</h4>
                    <small class="text-muted">Pending</small>
                </div>
                <div class="col-4">
                    <h4 class="mb-0 text-success">{{ $syncStats['synced'] ?? 0 }}</h4>
                    <small class="text-muted">Synced</small>
                </div>
                <div class="col-4">
                    <h4 class="mb-0 text-danger">{{ $syncStats['failed'] ?? 0 }}</h4>
                    <small class="text-muted">Failed</small>
                </div>
            </div>

            @if (isset($syncStats['last_sync']) && $syncStats['last_sync'])
                <p class="text-muted small mb-3">
                    Last sync: {{ $syncStats['last_sync']->diffForHumans() }}
                </p>
            @endif

            <div class="d-grid gap-2">
                <button wire:click="syncNow"
                        class="btn btn-primary"
                        @if($isSyncing || !($status['is_connected'] ?? false)) disabled @endif>
                    @if ($isSyncing)
                        <span class="spinner-border spinner-border-sm me-2"></span>
                        Syncing...
                    @else
                        <i class="bi bi-cloud-upload"></i>
                        Sync Now
                    @endif
```

```
                @endif
            </button>

            @if (($syncStats['failed'] ?? 0) > 0)
                <button wire:click="retryFailed" class="btn btn-outline-warning">
                    <i class="bi bi-arrow-repeat"></i>
                    Retry Failed ({{ $syncStats['failed'] }})
                </button>
            @endif

            @if (($syncStats['synced'] ?? 0) > 0)
                <button wire:click="clearSynced"
                        class="btn btn-outline-secondary btn-sm"
                        onclick="return confirm('Clear synced items from queue?')">
                    Clear Synced Items
                </button>
            @endif>
        </div>
    </div>
</div>

{{-- Network Tips --}}
<div class="card">
    <div class="card-header">
        <h6 class="mb-0">
            <i class="bi bi-info-circle"></i>
            How Sync Works
        </h6>
    </div>
    <div class="card-body">
        <ul class="mb-0">
            <li>Your data is saved locally first</li>
            <li>Changes are queued for sync when you're offline</li>
            <li>Auto-sync happens when connection is restored</li>
            <li>You can manually sync anytime</li>
            <li>Failed syncs will retry up to 3 times</li>
        </ul>
    </div>
</div>
</div>

<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-icons.css">
```

Step 9: Add Routes and Navigation

```
// routes/web.php
use App\Livewire\NetworkStatus;

Route::get('/network', NetworkStatus::class)->name('network');
```

Create a navigation component `resources/views/components/bottom-nav.blade.php`:

```
<nav class="navbar fixed-bottom navbar-light bg-white border-top">
    <div class="container-fluid">
        <div class="d-flex justify-content-around w-100">
            <a href="{{ route('home') }}" class="nav-link text-center">
                <i class="bi bi-house" style="font-size: 1.5rem;"></i>
                <div><small>Home</small></div>
            </a>
            <a href="{{ route('camera') }}" class="nav-link text-center">
                <i class="bi bi-camera" style="font-size: 1.5rem;"></i>
                <div><small>Camera</small></div>
            </a>
            <a href="{{ route('recorder') }}" class="nav-link text-center">
                <i class="bi bi-mic" style="font-size: 1.5rem;"></i>
                <div><small>Recorder</small></div>
            </a>
            <a href="{{ route('scanner') }}" class="nav-link text-center">
                <i class="bi bi-qr-code" style="font-size: 1.5rem;"></i>
                <div><small>Scanner</small></div>
            </a>
            <a href="{{ route('network') }}" class="nav-link text-center">
                <i class="bi bi-wifi" style="font-size: 1.5rem;"></i>
                <div><small>Network</small></div>
            </a>
        </div>
    </div>
</nav>
```

Complete Network Features:

- ✅ Real-time network status monitoring
- ✅ Connection type detection (WiFi, Cellular, None)
- ✅ Sync queue for offline operations
- ✅ Automatic sync when connection restored
- ✅ Manual sync trigger
- ✅ Retry failed syncs
- ✅ Sync statistics dashboard
- ✅ Connection quality indicators
- ✅ Error handling and logging
- ✅ Queue management (clear synced items)

Now I'll add the final parts with complete API integration and real-world examples. Continue?

Part 9: Complete API Integration Architecture

Overview

Build a production-ready API-first architecture with Laravel Sanctum authentication, file uploads, and real-time sync.

Backend API Setup

Step 1: Create Laravel Backend Project

```
# On your server/local machine for API
composer create-project laravel/laravel mobile-api
cd mobile-api
```

Step 2: Install Sanctum

```
composer require laravel/sanctum
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
php artisan migrate
```

Step 3: Configure Sanctum

Edit `app/Http/Kernel.php`:

```
'api' => [
    \Laravel\Sanctum\Http\Middleware\EnsureFrontendRequestsAreStateful::class,
    'throttle:api',
    \Illuminate\Routing\Middleware\SubstituteBindings::class,
],
```

Step 4: Create API Controllers

```
php artisan make:controller Api/AuthController
php artisan make:controller Api/PhotoController
php artisan make:controller Api/AudioRecordingController
```

AuthController.php:

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\ValidationException;

class AuthController extends Controller
{
    /**
     * Register new user
     */
    public function register(Request $request)
    {
        $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|string|email|max:255|unique:users',
            'password' => 'required|string|min:8|confirmed',
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
        ]);

        $token = $user->createToken('mobile-app')->plainTextToken;

        return response()->json([
            'user' => $user,
            'token' => $token,
        ], 201);
    }

    /**
     * Login user
     */
```

```
    public function login(Request $request)
    {
        $request->validate([
            'email' => 'required|email',
            'password' => 'required',
        ]);

        if (!Auth::attempt($request->only('email', 'password'))) {
            throw ValidationException::withMessages([
                'email' => ['The provided credentials are incorrect.'],
            ]);
        }

        $user = Auth::user();
        $token = $user->createToken('mobile-app')->plainTextToken;

        return response()->json([
            'user' => $user,
            'token' => $token,
        ]);
    }

    /**
     * Logout user
     */
    public function logout(Request $request)
    {
        $request->user()->currentAccessToken()->delete();

        return response()->json([
            'message' => 'Logged out successfully',
        ]);
    }

    /**
     * Get authenticated user
     */
    public function me(Request $request)
    {
        return response()->json($request->user());
    }
}
```

PhotoController.php:

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Photo;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Str;

class PhotoController extends Controller
{
    /**
     * Get all user's photos
     */
    public function index(Request $request)
    {
        $photos = $request->user()
            ->photos()
            ->latest('taken_at')
            ->get();
```

```
            ->get();

        return response()->json([
            'data' => $photos,
        ]);
    }

    /**
     * Store new photo
     */
    public function store(Request $request)
    {
        $request->validate([
            'title' => 'required|string|max:255',
            'description' => 'nullable|string',
            'image' => 'required|image|max:10240', // Max 10MB
            'taken_at' => 'required|date',
        ]);

        // Store image
        $file = $request->file('image');
        $filename = Str::uuid() . '.' . $file->extension();
        $path = $file->storeAs('photos/' . $request->user()->id, $filename, 'public');

        // Create photo record
        $photo = $request->user()->photos()->create([
            'title' => $request->title,
            'description' => $request->description,
            'path' => $path,
            'filename' => $filename,
            'size' => $file->getSize(),
            'mime_type' => $file->getMimeType(),
            'taken_at' => $request->taken_at,
            'remote_url' => Storage::url($path),
        ]);

        return response()->json([
            'data' => $photo,
            'message' => 'Photo uploaded successfully',
        ], 201);
    }

    /**
     * Get single photo
     */
    public function show(Request $request, Photo $photo)
    {
        // Ensure user owns the photo
        if ($photo->user_id !== $request->user()->id) {
            return response()->json([
                'message' => 'Unauthorized',
            ], 403);
        }

        return response()->json([
            'data' => $photo,
        ]);
    }

    /**
     * Update photo
     */
    public function update(Request $request, Photo $photo)
    {
        // Ensure user owns the photo
        if ($photo->user_id !== $request->user()->id) {
```

```
            return response()->json([
                'message' => 'Unauthorized',
            ], 403);
        }

        $request->validate([
            'title' => 'sometimes|string|max:255',
            'description' => 'nullable|string',
        ]);

        $photo->update($request->only(['title', 'description']));

        return response()->json([
            'data' => $photo,
            'message' => 'Photo updated successfully',
        ]);
    }

    /**
     * Delete photo
     */
    public function destroy(Request $request, Photo $photo)
    {
        // Ensure user owns the photo
        if ($photo->user_id !== $request->user()->id) {
            return response()->json([
                'message' => 'Unauthorized',
            ], 403);
        }

        // Delete file from storage
        if (Storage::disk('public')->exists($photo->path)) {
            Storage::disk('public')->delete($photo->path);
        }

        $photo->delete();

        return response()->json([
            'message' => 'Photo deleted successfully',
        ]);
    }
}
```

Step 5: Define API Routes

Edit `routes/api.php`:

```php
<?php

use App\Http\Controllers\Api\AuthController;
use App\Http\Controllers\Api\PhotoController;
use App\Http\Controllers\Api\AudioRecordingController;
use Illuminate\Support\Facades\Route;

// Public routes
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

// Protected routes
Route::middleware('auth:sanctum')->group(function () {
    // Auth
    Route::get('/me', [AuthController::class, 'me']);
    Route::post('/logout', [AuthController::class, 'logout']);

    // Photos
    Route::apiResource('photos', PhotoController::class);

    // Audio Recordings
    Route::apiResource('recordings', AudioRecordingController::class);
});
```

Step 6: Update Photo Model for User Relationship

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Photo extends Model
{
    protected $fillable = [
        'user_id',
        'title',
        'description',
        'path',
        'filename',
        'size',
        'mime_type',
        'width',
        'height',
        'taken_at',
        'location',
        'is_synced',
        'remote_url',
        'remote_id',
    ];

    protected $casts = [
        'taken_at' => 'datetime',
        'is_synced' => 'boolean',
    ];

    /**
     * Get the user that owns the photo
     */
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

Step 7: Add Photos Relationship to User Model

```php
<?php

namespace App\Models;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens;

    protected $fillable = [
        'name',
        'email',
        'password',
    ];

    protected $hidden = [
        'password',
        'remember_token',
    ];

    /**
     * Get user's photos
     */
    public function photos()
    {
        return $this->hasMany(Photo::class);
    }

    /**
     * Get user's recordings
     */
    public function recordings()
    {
        return $this->hasMany(AudioRecording::class);
    }
}
```

Mobile App API Integration

Step 1: Update API Service (Mobile App)

Update `app/Services/ApiService.php`:

```php
<?php

namespace App\Services;

use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Facades\Log;

class ApiService
{
    protected $baseUrl;
    protected $token;
    protected $timeout = 30;

    public function __construct()
    {
        $this->baseUrl = env('API_URL', 'http://localhost:8000');
        $this->token = Cache::get('auth_token');
    }
```

```
/**
 * Register new user
 */
public function register(string $name, string $email, string $password): ?array
{
    try {
        $response = Http::timeout($this->timeout)
            ->post($this->baseUrl . '/api/register', [
                'name' => $name,
                'email' => $email,
                'password' => $password,
                'password_confirmation' => $password,
            ]);

        if ($response->successful()) {
            $data = $response->json();
            $this->saveAuthData($data);
            return $data;
        }

        return null;
    } catch (\Exception $e) {
        Log::error('Registration failed: ' . $e->getMessage());
        return null;
    }
}

/**
 * Login user
 */
public function login(string $email, string $password): ?array
{
    try {
        $response = Http::timeout($this->timeout)
            ->post($this->baseUrl . '/api/login', [
                'email' => $email,
                'password' => $password,
            ]);

        if ($response->successful()) {
            $data = $response->json();
            $this->saveAuthData($data);
            return $data;
        }

        return null;
    } catch (\Exception $e) {
        Log::error('Login failed: ' . $e->getMessage());
        return null;
    }
}

/**
 * Logout user
 */
public function logout(): bool
{
    try {
        $response = Http::withToken($this->token)
            ->timeout($this->timeout)
            ->post($this->baseUrl . '/api/logout');

        $this->clearAuthData();
        return $response->successful();
```

```
        } catch (\Exception $e) {
            Log::error('Logout failed: ' . $e->getMessage());
            $this->clearAuthData();
            return false;
        }
    }

    /**
     * Get authenticated user
     */
    public function getUser(): ?array
    {
        try {
            $response = Http::withToken($this->token)
                ->timeout($this->timeout)
                ->get($this->baseUrl . '/api/me');

            if ($response->successful()) {
                return $response->json();
            }

            return null;
        } catch (\Exception $e) {
            Log::error('Get user failed: ' . $e->getMessage());
            return null;
        }
    }

    /**
     * Upload photo
     */
    public function uploadPhoto(string $filePath, string $title, ?string $description = null): ?array
    {
        try {
            $response = Http::withToken($this->token)
                ->timeout(60) // Longer timeout for file upload
                ->attach('image', file_get_contents($filePath), basename($filePath))
                ->post($this->baseUrl . '/api/photos', [
                    'title' => $title,
                    'description' => $description,
                    'taken_at' => now()->toIso8601String(),
                ]);

            if ($response->successful()) {
                return $response->json('data');
            }

            return null;
        } catch (\Exception $e) {
            Log::error('Photo upload failed: ' . $e->getMessage());
            return null;
        }
    }

    /**
     * Get all photos
     */
    public function getPhotos(): ?array
    {
        return $this->get('/api/photos');
    }

    /**
     * Update photo
     */
    public function updatePhoto(int $remoteId, array $data): ?array
```

```
{
    return $this->put("/api/photos/{$remoteId}", $data);
}

/**
 * Delete photo
 */
public function deletePhoto(int $remoteId): bool
{
    return $this->delete("/api/photos/{$remoteId}");
}

/**
 * Generic GET request
 */
public function get(string $endpoint): ?array
{
    try {
        $response = Http::withToken($this->token)
            ->timeout($this->timeout)
            ->get($this->baseUrl . $endpoint);

        if ($response->successful()) {
            return $response->json();
        }

        return null;
    } catch (\Exception $e) {
        Log::error("GET {$endpoint} failed: " . $e->getMessage());
        throw $e;
    }
}

/**
 * Generic POST request
 */
public function post(string $endpoint, array $data): ?array
{
    try {
        $response = Http::withToken($this->token)
            ->timeout($this->timeout)
            ->post($this->baseUrl . $endpoint, $data);

        if ($response->successful()) {
            return $response->json();
        }

        return null;
    } catch (\Exception $e) {
        Log::error("POST {$endpoint} failed: " . $e->getMessage());
        throw $e;
    }
}

/**
 * Generic PUT request
 */
public function put(string $endpoint, array $data): ?array
{
    try {
        $response = Http::withToken($this->token)
            ->timeout($this->timeout)
            ->put($this->baseUrl . $endpoint, $data);

        if ($response->successful()) {
```

```
                return $response->json();
            }

            return null;
        } catch (\Exception $e) {
            Log::error("PUT {$endpoint} failed: " . $e->getMessage());
            throw $e;
        }
    }

    /**
     * Generic DELETE request
     */
    public function delete(string $endpoint): bool
    {
        try {
            $response = Http::withToken($this->token)
                ->timeout($this->timeout)
                ->delete($this->baseUrl . $endpoint);

            return $response->successful();
        } catch (\Exception $e) {
            Log::error("DELETE {$endpoint} failed: " . $e->getMessage());
            throw $e;
        }
    }

    /**
     * Save authentication data
     */
    private function saveAuthData(array $data): void
    {
        Cache::put('auth_token', $data['token'], now()->addDays(30));
        Cache::put('auth_user', $data['user'], now()->addDays(30));
    }

    /**
     * Clear authentication data
     */
    private function clearAuthData(): void
    {
        Cache::forget('auth_token');
        Cache::forget('auth_user');
    }

    /**
     * Check if user is authenticated
     */
    public function isAuthenticated(): bool
    {
        return !empty($this->token);
    }

    /**
     * Get cached user data
     */
    public function getCachedUser(): ?array
    {
        return Cache::get('auth_user');
    }
}
```

Step 2: Create Auth Component

```
php artisan make:livewire Auth/Login
```

```
<?php

namespace App\Livewire\Auth;

use Livewire\Component;
use App\Services\ApiService;

class Login extends Component
{
    public $email = '';
    public $password = '';
    public $name = '';
    public $isRegistering = false;
    public $rememberMe = false;

    protected $apiService;

    public function boot(ApiService $apiService)
    {
        $this->apiService = $apiService;
    }

    public function mount()
    {
        // Redirect if already logged in
        if ($this->apiService->isAuthenticated()) {
            return redirect()->route('home');
        }
    }

    /**
     * Login user
     */
    public function login()
    {
        $this->validate([
            'email' => 'required|email',
            'password' => 'required|min:8',
        ]);

        $result = $this->apiService->login($this->email, $this->password);

        if ($result) {
            session()->flash('success', 'Welcome back!');
            return redirect()->route('home');
        }

        $this->addError('email', 'Invalid credentials. Please try again.');
    }

    /**
     * Register new user
     */
    public function register()
    {
        $this->validate([
            'name' => 'required|string|min:3',
            'email' => 'required|email',
            'password' => 'required|min:8',
        ]);

        $result = $this->apiService->register($this->name, $this->email, $this->password);
```

```
        if ($result) {
            session()->flash('success', 'Account created successfully!');
            return redirect()->route('home');
        }

        $this->addError('email', 'Registration failed. Email may already be in use.');
    }

    /**
     * Toggle between login and register
     */
    public function toggleMode()
    {
        $this->isRegistering = !$this->isRegistering;
        $this->resetErrorBag();
        $this->resetValidation();
    }

    public function render()
    {
        return view('livewire.auth.login')->layout('layouts.guest');
    }
}
```

Create `resources/views/livewire/auth/login.blade.php`:

```
<div class="container">
    <div class="row justify-content-center align-items-center min-vh-100">
        <div class="col-md-8 col-lg-6">
            <div class="text-center mb-4">
                <h1 class="display-4">{{ config('app.name') }}</h1>
                <p class="text-muted">Your mobile companion</p>
            </div>

            <div class="card">
                <div class="card-body p-4">
                    <h3 class="card-title mb-4 text-center">
                        {{ $isRegistering ? 'Create Account' : 'Welcome Back' }}
                    </h3>

                    <form wire:submit.prevent="{{ $isRegistering ? 'register' : 'login' }}">
                        @if ($isRegistering)
                            <div class="mb-3">
                                <label class="form-label">Name</label>
                                <input type="text"
                                       wire:model="name"
                                       class="form-control @error('name') is-invalid @enderror"
                                       placeholder="Your name">
                                @error('name')
                                    <div class="invalid-feedback">{{ $message }}</div>
                                @enderror
                            </div>
                        @endif

                        <div class="mb-3">
                            <label class="form-label">Email</label>
                            <input type="email"
                                   wire:model="email"
                                   class="form-control @error('email') is-invalid @enderror"
                                   placeholder="your@email.com">
                            @error('email')
                                <div class="invalid-feedback">{{ $message }}</div>
                            @enderror
                        </div>
```

```
                    <div class="mb-3">
                        <label class="form-label">Password</label>
                        <input type="password"
                               wire:model="password"
                               class="form-control @error('password') is-invalid @enderror"
                               placeholder="••••••••">
                        @error('password')
                            <div class="invalid-feedback">{{ $message }}</div>
                        @enderror
                    </div>

                    @if (!$isRegistering)
                        <div class="mb-3 form-check">
                            <input type="checkbox"
                                   wire:model="rememberMe"
                                   class="form-check-input"
                                   id="rememberMe">
                            <label class="form-check-label" for="rememberMe">
                                Remember me
                            </label>
                        </div>
                    @endif

                    <div class="d-grid mb-3">
                        <button type="submit" class="btn btn-primary btn-lg">
                            {{ $isRegistering ? 'Create Account' : 'Login' }}
                        </button>
                    </div>
                </form>

                <div class="text-center">
                    <button wire:click="toggleMode" class="btn btn-link">
                        {{ $isRegistering ? 'Already have an account? Login' : "Don't have an account? Register" }}
                    </button>
                </div>
            </div>
        </div>
    </div>
</div>
```

Step 3: Create Auth Middleware

```
php artisan make:middleware EnsureAuthenticated
```

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use App\Services\ApiService;

class EnsureAuthenticated
{
    protected $apiService;

    public function __construct(ApiService $apiService)
    {
        $this->apiService = $apiService;
    }

    public function handle(Request $request, Closure $next)
    {
        if (!$this->apiService->isAuthenticated()) {
            return redirect()->route('login');
        }

        return $next($request);
    }
}
```

Register middleware in `app/Http/Kernel.php`:

```php
protected $routeMiddleware = [
    // ... other middleware
    'mobile.auth' => \App\Http\Middleware\EnsureAuthenticated::class,
];
```

Update routes in `routes/web.php`:

```php
use App\Livewire\Auth\Login;

// Guest routes
Route::get('/login', Login::class)->name('login');

// Protected routes
Route::middleware('mobile.auth')->group(function () {
    Route::get('/', Home::class)->name('home');
    Route::get('/camera', CameraCapture::class)->name('camera');
    Route::get('/recorder', VoiceRecorder::class)->name('recorder');
    Route::get('/scanner', BarcodeScanner::class)->name('scanner');
    Route::get('/network', NetworkStatus::class)->name('network');
    Route::get('/files', FileManager::class)->name('files');
});
```

Complete API Integration Features:

- ✅ User registration and authentication
- ✅ Token-based auth with Laravel Sanctum
- ✅ Photo upload with multipart/form-data
- ✅ CRUD operations for photos
- ✅ User relationships
- ✅ Secure API endpoints
- ✅ File storage on server
- ✅ Error handling and logging
- ✅ Auth middleware protection
- ✅ Automatic token management

Shall I continue with Part 10 (Real-World Example: Complete Photo Gallery App) and Part 11 (Real-World Example: Task Manager with API)?

Part 10: Real-World Example - Complete Instagram-Style Photo Gallery App

Project Overview

Build a full-featured photo gallery app with filters, social features, and cloud sync.

Features

- Photo capture with filters
- Grid and list view
- Albums/Collections
- Search and tags
- Share photos
- Like and comment
- Cloud backup
- Offline mode

Implementation

Database Schema

```
php artisan make:migration create_albums_table
php artisan make:migration create_photo_album_table
php artisan make:migration create_photo_likes_table
php artisan make:migration create_photo_comments_table
```

Albums Migration:

```
Schema::create('albums', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->string('name');
    $table->text('description')->nullable();
    $table->string('cover_photo_id')->nullable();
    $table->boolean('is_public')->default(false);
    $table->timestamps();
});
```

Photo-Album Pivot:

```
Schema::create('photo_album', function (Blueprint $table) {
    $table->foreignId('photo_id')->constrained()->cascadeOnDelete();
    $table->foreignId('album_id')->constrained()->cascadeOnDelete();
    $table->integer('order')->default(0);
    $table->timestamps();
});
```

Complete PhotoGallery Component

```
<?php

namespace App\Livewire;

use Livewire\Component;
use App\Models\Photo;
use App\Models\Album;
use App\Services\PhotoService;
use App\Services\ApiService;
```

```
class PhotoGallery extends Component
{
    public $photos = [];
    public $albums = [];
    public $currentView = 'grid'; // grid or list
    public $selectedAlbum = null;
    public $searchQuery = '';
    public $filterMode = 'all'; // all, favorites, recent

    // Create album
    public $showCreateAlbum = false;
    public $newAlbumName = '';
    public $newAlbumDescription = '';

    // Photo selection for bulk operations
    public $selectedPhotos = [];
    public $selectionMode = false;

    protected $photoService;
    protected $apiService;

    public function boot(PhotoService $photoService, ApiService $apiService)
    {
        $this->photoService = $photoService;
        $this->apiService = $apiService;
    }

    public function mount()
    {
        $this->loadPhotos();
        $this->loadAlbums();
    }

    public function loadPhotos()
    {
        $query = Photo::query();

        if ($this->selectedAlbum) {
            $query->whereHas('albums', function($q) {
                $q->where('album_id', $this->selectedAlbum);
            });
        }

        if ($this->searchQuery) {
            $query->where(function($q) {
                $q->where('title', 'like', "%{$this->searchQuery}%")
                  ->orWhere('description', 'like', "%{$this->searchQuery}%");
            });
        }

        switch ($this->filterMode) {
            case 'favorites':
                $query->where('is_favorite', true);
                break;
            case 'recent':
                $query->where('taken_at', '>=', now()->subDays(7));
                break;
        }

        $this->photos = $query->latest('taken_at')->get();
    }

    public function loadAlbums()
    {
        $this->albums = Album::withCount('photos')->get();
```

```
    }

    public function createAlbum()
    {
        $this->validate([
            'newAlbumName' => 'required|string|max:255',
        ]);

        Album::create([
            'user_id' => auth()->id() ?? 1, // Fallback for demo
            'name' => $this->newAlbumName,
            'description' => $this->newAlbumDescription,
        ]);

        $this->loadAlbums();
        $this->showCreateAlbum = false;
        $this->newAlbumName = '';
        $this->newAlbumDescription = '';

        session()->flash('success', 'Album created!');
    }

    public function selectAlbum($albumId)
    {
        $this->selectedAlbum = $albumId;
        $this->loadPhotos();
    }

    public function clearAlbumFilter()
    {
        $this->selectedAlbum = null;
        $this->loadPhotos();
    }

    public function toggleView()
    {
        $this->currentView = $this->currentView === 'grid' ? 'list' : 'grid';
    }

    public function toggleSelectionMode()
    {
        $this->selectionMode = !$this->selectionMode;
        if (!$this->selectionMode) {
            $this->selectedPhotos = [];
        }
    }

    public function togglePhotoSelection($photoId)
    {
        if (in_array($photoId, $this->selectedPhotos)) {
            $this->selectedPhotos = array_diff($this->selectedPhotos, [$photoId]);
        } else {
            $this->selectedPhotos[] = $photoId;
        }
    }

    public function addSelectedToAlbum($albumId)
    {
        $album = Album::find($albumId);
        $album->photos()->syncWithoutDetaching($this->selectedPhotos);

        $count = count($this->selectedPhotos);
        $this->selectedPhotos = [];
        $this->selectionMode = false;

        session()->flash('success', "{$count} photos added to album!");
```

```
        session()->flash('success', "{$count} photos added to album!");
    }

    public function deleteSelected()
    {
        foreach ($this->selectedPhotos as $photoId) {
            $photo = Photo::find($photoId);
            if ($photo) {
                $this->photoService->deletePhoto($photo);
            }
        }

        $count = count($this->selectedPhotos);
        $this->selectedPhotos = [];
        $this->selectionMode = false;
        $this->loadPhotos();

        session()->flash('success', "{$count} photos deleted!");
    }

    public function syncToCloud()
    {
        $unsyncedPhotos = Photo::where('is_synced', false)->get();

        $synced = 0;
        foreach ($unsyncedPhotos as $photo) {
            try {
                $result = $this->apiService->uploadPhoto(
                    $photo->full_path,
                    $photo->title,
                    $photo->description
                );

                if ($result) {
                    $photo->update([
                        'is_synced' => true,
                        'remote_id' => $result['id'],
                        'remote_url' => $result['remote_url'],
                    ]);
                    $synced++;
                }
            } catch (\Exception $e) {
                // Continue with next photo
                continue;
            }
        }

        session()->flash('success', "Synced {$synced} photos to cloud!");
    }

    public function render()
    {
        return view('livewire.photo-gallery');
    }
}
```

Album Model

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Album extends Model
{
    protected $fillable = [
        'user_id',
        'name',
        'description',
        'cover_photo_id',
        'is_public',
    ];

    protected $casts = [
        'is_public' => 'boolean',
    ];

    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function photos()
    {
        return $this->belongsToMany(Photo::class)
            ->withPivot('order')
            ->withTimestamps()
            ->orderBy('pivot_order');
    }

    public function coverPhoto()
    {
        return $this->belongsTo(Photo::class, 'cover_photo_id');
    }
}
```

Part 11: Publishing & Production Deployment

Preparing for Production

Step 1: Update Configuration

```
// config/nativephp.php

return [
    'app_id' => env('NATIVEPHP_APP_ID', 'com.yourcompany.photoapp'),
    'app_version' => env('NATIVEPHP_APP_VERSION', '1.0.0'),
    'version_code' => 1, // Increment for each release
    'name' => env('NATIVEPHP_APP_NAME', 'PhotoApp'),

    'android' => [
        'minSdkVersion' => 24,
        'targetSdkVersion' => 34,
        'compileSdkVersion' => 34,

        // Production signing
        'signing' => [
            'storeFile' => env('ANDROID_KEYSTORE_PATH'),
            'storePassword' => env('ANDROID_KEYSTORE_PASSWORD'),
            'keyAlias' => env('ANDROID_KEY_ALIAS'),
            'keyPassword' => env('ANDROID_KEY_PASSWORD'),
        ],
    ],

    'ios' => [
        'minVersion' => '13.0',
        'teamId' => env('APPLE_TEAM_ID'),
        'bundleId' => env('APPLE_BUNDLE_ID'),
    ],

    // App icons (required sizes)
    'icons' => [
        'android' => [
            'mdpi' => resource_path('images/icons/android/mdpi.png'), // 48x48
            'hdpi' => resource_path('images/icons/android/hdpi.png'), // 72x72
            'xhdpi' => resource_path('images/icons/android/xhdpi.png'), // 96x96
            'xxhdpi' => resource_path('images/icons/android/xxhdpi.png'), // 144x144
            'xxxhdpi' => resource_path('images/icons/android/xxxhdpi.png'), // 192x192
        ],
        'ios' => [
            'AppIcon' => resource_path('images/icons/ios/AppIcon.appiconset'),
        ],
    ],
];
```

Step 2: Create App Icons

Use a tool like AppIcon.co to generate all required sizes.

Step 3: Create Privacy Policy & Terms

Required for both App Store and Play Store:

```
// routes/web.php
Route::get('/privacy-policy', function () {
    return view('legal.privacy');
})->name('privacy');

Route::get('/terms-of-service', function () {
    return view('legal.terms');
})->name('terms');
```

Step 4: Optimize for Production

```
# Optimize autoloader
composer install --optimize-autoloader --no-dev

# Cache configuration
php artisan config:cache

# Cache routes
php artisan route:cache

# Cache views
php artisan view:cache

# Minify assets
npm run build
```

Step 5: Test Production Build

```
# Build release APK
php artisan native:build android --release

# Test on physical device
adb install -r nativephp/android/app/build/outputs/apk/release/app-release.apk
```

Publishing to Google Play Store

Step 1: Create Play Console Account

1. Go to Google Play Console
2. Pay $25 one-time registration fee
3. Complete account setup

Step 2: Create New App

1. Click "Create app"
2. Fill in:
    - App name
    - Default language
    - App or game
    - Free or paid
3. Accept declarations

Step 3: Set Up Store Listing

Required:

- App name
- Short description (80 characters)
- Full description (4000 characters)
- App icon (512 x 512 PNG)
- Feature graphic (1024 x 500 PNG)
- Screenshots (minimum 2, maximum 8)
- Category
- Contact email
- Privacy policy URL

Example Description:

```
Short Description:
Capture, organize, and share your photos with powerful native features.

Full Description:
PhotoApp is your complete mobile photography solution:

 CAPTURE
• High-quality photo capture
• Advanced camera controls
• Instant filters and effects

 ORGANIZE
• Create unlimited albums
• Smart search and tags
• Auto-categorization

☁ SYNC
• Automatic cloud backup
• Access anywhere
• Secure encryption

 EDIT
• Powerful editing tools
• Filters and adjustments
• Crop, rotate, enhance

Share your moments with friends and family. Download now!
```

Step 4: Upload App Bundle

1. Go to "Production" → "Create new release"
2. Upload your AAB or APK
3. Add release notes
4. Review and roll out

Step 5: Complete Content Rating

Answer questionnaire about app content:

- Violence
- Sexual content
- Profanity
- Controlled substances
- etc.

Step 6: Set Up Pricing & Distribution

- Select countries
- Set price (or free)
- Add content guidelines
- Review privacy policy

Step 7: Submit for Review

Review process typically takes 3-7 days.

Publishing to Apple App Store

Step 1: Enroll in Apple Developer Program

- Cost: $99/year
- Visit developer.apple.com

Step 2: Create App ID

1. Go to Certificates, Identifiers & Profiles
2. Click Identifiers → Add
3. Enter Bundle ID (e.g., com.yourcompany.photoapp)
4. Enable capabilities:

- Push Notifications
- iCloud (if using)
- etc.

Step 3: Create App in App Store Connect

1. Go to App Store Connect
2. Click "My Apps" → "+"
3. Fill in:
    - Platform: iOS
    - Name
    - Primary Language
    - Bundle ID
    - SKU

Step 4: Build and Archive

```
# Build iOS app
php artisan native:build ios --release

# Open in Xcode
open nativephp/ios/YourApp.xcworkspace

# In Xcode:
# 1. Select "Any iOS Device"
# 2. Product → Archive
# 3. Upload to App Store Connect
```

Step 5: Complete App Information

Required:

- App name
- Subtitle (30 characters)
- Category (Primary & Secondary)
- Screenshots for all device sizes:
    - 6.7" (iPhone 14 Pro Max)
    - 6.5" (iPhone 11 Pro Max)
    - 5.5" (iPhone 8 Plus)
    - iPad Pro (12.9")
- Description (4000 characters)
- Keywords (100 characters)
- Support URL
- Privacy Policy URL

Step 6: Submit for Review

Apple review typically takes 24-48 hours.

Post-Launch Monitoring

Analytics Setup

```
composer require laravel/telescope --dev
php artisan telescope:install
php artisan migrate
```

Crash Reporting

Integrate Sentry for crash tracking:

```
composer require sentry/sentry-laravel
php artisan sentry:publish --dsn=YOUR_DSN
```

User Feedback

Add in-app feedback:

```
// In your app
Route::post('/api/feedback', function (Request $request) {
    // Store feedback
    Feedback::create([
        'user_id' => $request->user()->id,
        'message' => $request->message,
        'rating' => $request->rating,
        'device_info' => $request->device_info,
    ]);

    return response()->json(['message' => 'Thank you!']);
});
```

Part 12: Best Practices & Tips

Performance Optimization

1. Image Optimization

```
// Compress images before saving
public function optimizeImage(string $path): void
{
    $image = Image::make($path);

    // Resize large images
    if ($image->width() > 1920) {
        $image->resize(1920, null, function ($constraint) {
            $constraint->aspectRatio();
            $constraint->upsize();
        });
    }

    // Compress
    $image->save($path, 80); // 80% quality
}
```

2. Database Indexing

```
// In migrations
$table->index('user_id');
$table->index('created_at');
$table->index(['user_id', 'created_at']);
```

3. Lazy Loading

```
// Load photos with pagination
public function loadPhotos()
{
    $this->photos = Photo::latest()
        ->limit(20)
        ->offset($this->page * 20)
        ->get();
}
```

Security Best Practices

1. API Security

```
// Rate limiting
Route::middleware('throttle:60,1')->group(function () {
    // 60 requests per minute
});

// Input validation
$request->validate([
    'email' => 'required|email|max:255',
    'password' => 'required|min:8|regex:/[a-z]/|regex:/[A-Z]/|regex:/[0-9]/',
]);
```

2. Secure File Storage

```
// Never expose direct paths
// Use signed URLs instead
$url = Storage::temporaryUrl('photos/image.jpg', now()->addMinutes(5));
```

3. SQL Injection Prevention

```
// Always use parameter binding
$photos = DB::select('SELECT * FROM photos WHERE user_id = ?', [$userId]);

// Or use Query Builder/Eloquent
$photos = Photo::where('user_id', $userId)->get();
```

Testing

Unit Tests

```
// tests/Unit/PhotoServiceTest.php
class PhotoServiceTest extends TestCase
{
    public function test_can_save_photo()
    {
        $service = new PhotoService();
        $photo = $service->savePhoto('/path/to/test.jpg', 'Test Photo');

        $this->assertInstanceOf(Photo::class, $photo);
        $this->assertEquals('Test Photo', $photo->title);
    }
}
```

Feature Tests

```
// tests/Feature/PhotoUploadTest.php
class PhotoUploadTest extends TestCase
{
    public function test_authenticated_user_can_upload_photo()
    {
        $user = User::factory()->create();

        $response = $this->actingAs($user)
            ->post('/api/photos', [
                'title' => 'Test Photo',
                'image' => UploadedFile::fake()->image('photo.jpg'),
            ]);

        $response->assertStatus(201);
        $this->assertDatabaseHas('photos', ['title' => 'Test Photo']);
    }
}
```

Conclusion

What You've Built

Congratulations! You now have:

 Complete mobile app development knowledge Multiple working features (Camera, Audio, Scanner, Files, Network) Full API integration with authentication Production-ready sync system Real-world application examples Publishing knowledge for both platforms

Next Steps

1. **Build Your App**: Start with one feature and expand
2. **Join Community**: Connect with other NativePHP developers
3. **Contribute**: Share your plugins and experiences
4. **Stay Updated**: Follow NativePHP releases and updates

Resources

- **Official Docs**: https://nativephp.com/docs
- **GitHub**: https://github.com/NativePHP
- **Discord**: Join the community
- **This Documentation**: Keep it as reference!

Final Tips

1. Start small - implement one feature at a time
2. Test on real devices frequently
3. Monitor app performance and crashes
4. Listen to user feedback
5. Keep dependencies updated
6. Back up your code and keystores
7. Read app store guidelines carefully
8. Plan for updates and maintenance

**Happy building! **

End of Complete NativePHP Mobile Documentation

Created with ❤ for the Laravel & NativePHP Community
